BioPim Library Guide

Erwan Drézen

March 19, 2026

Contents

1 First steps **7**

1.1 A first example . . . 7

1.1.1 Task and launcher . . . 7

1.1.2 Parallelization . . . 10

1.2 Getting and using the *BPL* . . . 12

1.2.1 Requirements . . . 12

1.2.2 Getting the *BPL* . . . 12

1.2.3 Creating a project from scratch . . . 12

1.2.4 Compilation toolchain with the *BPL* . . . 13

1.2.5 Implementing a task with the *BPL* . . . 14

2 The design of the BPL **17**

2.1 Introduction . . . 17

2.2 Task, Launcher and architectures . . . 17

2.2.1 ArchUpmem . . . 21

2.2.2 ArchMulticore . . . 22

2.3 The "split" process in the BPL . . . 23

2.4 Reducing the results . . . 28

2.5 Serialization . . . 30

2.6 DPU source code . . . 31

2.7 Tagging classes for behaviour customization . . . 33

2.7.1 Why tags? . . . 33

2.7.2 Tag once . . . 33

2.7.3 Tag global . . . 35

2.7.4 Tag glonce . . . 36

2.8 The genesis of the bpl::vector class . . . 36

2.8.1 Introduction . . . 36

2.8.2 The bpl::vector class and its companion bpl::MemoryTree . . . 38

2.8.3 The vector_view class . . . 42

2.8.4 Cache size tuning . . . 43

2.8.5 Iterators . . . 44
2.8.6 Swapping items . . . 45
2.8.7 Performance . . . 46
2.9 Customization of the BPL . . . 48
2.10 The CMake toolchain . . . 48
2.11 Metaprogramming in the BPL . . . 49

3 Testing the BPL **51**
3.1 Unit tests . . . 51
3.1.1 Introduction . . . 51
3.1.2 Tests categories . . . 51
3.1.3 Unit test framework . . . 52
3.1.4 Source code hierarchy . . . 53
3.1.5 Building and running the unit tests . . . 53
3.2 Benchmark tests . . . 53
3.2.1 Introduction . . . 53
3.2.2 The benchmark report . . . 54
3.2.3 Annotation of a test . . . 56
3.2.4 Building and running the benchmark tests . . . 57
3.2.5 Other remarks . . . 57
3.2.6 Future improvements . . . 58
3.3 The LOC metric . . . 58
3.3.1 Example . . . 58

Introduction

The purpose of this document is to present the *BioPim Library* (also known as *BPL*) to the end user, enabling the design and implementation of programs that can run on different hardware architectures, such as multicore and *PIM* architectures like the *UPMEM* architecture.

The underlying idea is also to accelerate the development process, particularly when targeting the UPMEM architecture, by encapsulating all low-level SDK calls within the library. The expected advantage is to avoid discouraging developers with what might be perceived as an overly low-level SDK, thereby reaching a broader developer community.

This document also outlines the compilation toolchain requirements and provides the fundamentals for creating programs from scratch.

The *BPL* is written in C++, and therefore programs that use it should also be written in this language

Chapter 1

First steps

1.1 A first example

1.1.1 Task and launcher

As an introduction, here is a small program using the *BPL* that illustrates the overall philosophy of the library. This program simply computes the sum of the elements of an input integer vector. To do so, one must implement a struct or class that provides a method performing the actual computation; this method is an overload of the operator() function. In the following, we will often refer to such a structure as a **task**. In this example, the task is a structure named Sum.

```
template<typename ARCH>
struct Sum
{
    USING(ARCH);

    int32_t operator() (vector<int32_t> const& in) const
    {
        int32_t result = 0 ;
        for (auto n : in)  { result += n; }
        return result ;
    }
} ;
```

For the moment, we do not discuss the USING macro, which will be explained later.

A few important preliminary remarks:

- The Sum struct must be templated with a parameter ARCH that represents the hardware architecture on which the task will be executed. As a consequence, the implementation of the task should be agnostic with respect to the actual architecture that will ultimately be used. Following this approach, the task can, in theory, be executed on potentially very different kinds of hardware architectures.
- The code itself is rather straightforward: it receives an in vector as an argument, which is iterated over while updating the overall sum of its elements. The important

point here is that the only data[1] that can be processed by the task are the input arguments of the operator() method; for instance, there is no way to retrieve data from a file in a filesystem. On the other hand, the body of the task may define new objects (allocated on the execution stack) that can be returned if needed, such as the result variable holding the output of the task.

- Since we aim to design fast algorithms, we may want to leverage parallelization features such as threads on a multicore architecture. Note that, from the task's point of view (here, our Sum struct), there is no reference to threads or other parallelization mechanisms; the parallelization scheme is handled at another level. In other words, the task is agnostic with respect to its own parallelization: it simply receives input data, processes it, and returns a result.

- The *BPL* requires that the task (the Sum struct in our example) be placed in a header file following a specific naming convention. That is, the header file must be named X.hpp if the structure is named X. In our example, the header file must therefore be named Sum.hpp.

Once we have written our Sum task, the *BPL* provides mechanisms to use it, that is, to invoke it and retrieve a result. To do so, the *BPL* introduces the concept of a **launcher**, an object that takes as parameters a task (via the name of the structure that implements its logic) and the input arguments to be processed.

This is achieved through the bpl::Launcher class. This class is responsible for handling the hardware architecture to be used; therefore, when creating a launcher, it must be instantiated with a template argument representing the target hardware architecture. For instance, if we want to use a multicore architecture, we would write

```
Launcher<ArchMulticore> launcher;
```

If we want to use an *UPMEM* architecture, we would write

```
Launcher<ArchUpmem> launcher;
```

At this point, we are done with specifying the hardware architecture to be used: it is defined through the instantiation of the Launcher, and only there.

To run our Sum task, we use the run method of the Launcher class.

```
vector<int32_t> myvector = {1,2,3,5,8};
auto results = launcher.run<Sum> (myvector);
```

Note the following:

- The task to be executed is provided as a template argument to the Launcher::run method.

- The arguments (myvector in the example) are passed as parameters to the instantiated version of Launcher::run.

[1] There may be some exceptions to this remark.

- The result of the execution of Sum::operator() is retrieved in the results variable. At this point, note the *s* in results: it turns out that multiple results can be retrieved, rather than a single one (i.e., a simple int32_t in our Sum example).

To run this example, we can put everything into a main.cpp file containing the following code:

```
#include <bpl/bpl.hpp>
#include <Sum.hpp>
#include <vector>

int main()
{
    std::vector<int32_t> myvector = {1,2,3,5,8};

    bpl::Launcher<bpl::ArchMulticore> launcher;
    auto results = launcher.run<Sum> (myvector);

    return 0 ;
}
```

Note the inclusion of the bpl.hpp file to use the API of the Launcher class. We also need to include the Sum.hpp file, where our task is defined; indeed, this definition is required in order to call the Launcher::run method with Sum as a template argument.

A one-liner version would be

```
#include <bpl/bpl.hpp>
#include <Sum.hpp>
#include <vector>

int main()
{
    auto results = bpl::Launcher<bpl::ArchMulticore>{}.run<Sum> (
        std::vector<int32_t> {1,2,3,5,8}
    );
    return 0 ;
}
```

Note that we could have used a different architecture when declaring our launcher, for instance by using ArchUpmem instead:

```
#include <bpl/bpl.hpp>
#include <Sum.hpp>
#include <vector>

int main()
{
    auto results = bpl::Launcher<bpl::ArchUpmem>{}.run<Sum> (
        std::vector<int32_t> {1,2,3,5,8});
```

```
    return 0 ;
}
```

Now, our program can be compiled using the CMake tool. In fact, the *BPL* provides a compilation toolchain that hides many low-level details; this toolchain is an important part of the *BPL* and will be described in detail later.

From the end-user's perspective, everything we have seen so far resembles regular C++ development. We implement a small algorithm (summing the items of a vector) as a functor—that is, a struct with an overloaded operator() method. We then use the bpl::Launcher class to run it. Nothing in our code is specific to low-level details of the *UPMEM* architecture, for example; everything is handled by the *BPL* itself. On the other hand, we have not yet discussed how to parallelize our algorithm to take full advantage of the underlying hardware architecture.

1.1.2 Parallelization

Modern hardware architectures can greatly speed up programs by parallelizing algorithms; for example, by splitting an algorithm into small pieces that can be processed simultaneously and then aggregating the final result from the results of each piece.

In multicore architectures, this is often achieved using the well-known concept of a thread. Broadly speaking, using N threads can potentially divide the execution time of an algorithm by N, since each thread can run on a specific CPU of the underlying architecture. On the other hand, the *UPMEM* architecture uses the concept of a tasklet, which is similar at first glance to a thread but with significant differences that we will examine later. For now, we introduce the concept of a **process unit** to unify both concepts of thread and tasklet—that is, an entity that can execute a portion of a program on part of the underlying hardware architecture.

One of the main challenges in designing the *BPL* was to allow developers to write a single codebase that can run on different architectures (currently mainly multicore CPUs and *UPMEM*). This means that, when implementing a task, the underlying architecture is known only through the template parameter ARCH, so no assumptions should be made about a specific architecture. It is only when a bpl::Launcher is declared with a specific architecture and the Launcher::run method is called for the target class that all the specifics of the underlying architecture are taken into account.

Consequently, the *BPL* design had to accommodate different parallelization schemes, both for multicore and *UPMEM* environments. This is the role of the bpl::Launcher class: a launcher not only knows about the underlying architecture but also knows how to parallelize the task for the target architecture.

Another consideration is how many resources we want to allocate to run our task. On a 32-core architecture, for example, we might want to use only 8 cores. This can be achieved by providing the number of process units (threads, in this case) to use as an argument to the constructor of bpl::Launcher.

```
Launcher<ArchMulticore> launcher (Thread{8});
```

which can also be writen with

```
// launcher will use 8 process units (i.e. threads)
Launcher<ArchMulticore> launcher (8_thread);
```

So 8 threads will be used when calling `launcher.run<Sum>(myvector)`, each performing the same summation on the input vector.

When using the *UPMEM* architecture, it is possible to specify the amount of resources by setting the number of ranks or DPUs, e.g.,

```
// launcher1 will use 20*64*16 process units (i.e. tasklets)
Launcher<ArchUpmem> launcher1 (20_rank);
// launcher2 will use 500*16 process units (i.e. tasklets)
Launcher<ArchUpmem> launcher2 (500_dpu);
```

What we have just done is define how many resources a launcher can use and the corresponding number of process units running the task. However, each process unit will process the same input data and therefore produce the same result. We now need to specify how to parallelize the algorithm. The *BPL* provides a solution by splitting the input arguments of the task and assigning each specific split to a dedicated process unit. In our example, the input vector in will be divided into sub-vectors, and each sub-vector (potentially empty) will be provided as an argument to a separate instantiation of the Sum structure—one per process unit. This parallelization scheme can be easily applied by wrapping the myvector argument with the bpl::split method.

```
auto results = launcher.run<Sum> (split(myvector));
```

Now, a word about the 's' in results: each process unit runs an instance of Sum and produces a result. Therefore, if we have N process units, results will be a vector of N `int32_t` objects[2]. Thus, we could write

```
// Iterate over the result of each process unit
for (auto result : launcher.run<Sum> (split(myvector)))
{
    /* do something with result */
}
```

We will see later that bpl::split can be (template) parameterized to achieve more specific behavior. For now, we just keep in mind the default parallelization model of the *BPL* : the task itself does not know about parallelization; it is up to the end user (through the launcher) to provide a specific portion of the data that will feed each instance of the task running on a given process unit.

Note that this model is quite simple and may not be sufficient for every parallelization scheme. Other models could be proposed in the future.

We have now briefly covered some of the main concepts of the *BPL* , such as tasks and launchers. Next, we will go a step further by retrieving the *BPL* , creating a new *BPL* project, and compiling it.

[2]Note that results could be any iterable type rather than a full-fledged vector.

1.2 Getting and using the *BPL*

1.2.1 Requirements

Since one of the main target architectures of the *BPL* is the *UPMEM* architecture, you should install the *UPMEM* SDK. In particular, the *BPL* works with version 2023.2 (see `https://sdk.upmem.com`), so it is recommended to use this version.

Once installed, you need to define the `UPMEM_HOME` environment variable to point to the *UPMEM* SDK, for example:

```
UPMEM_HOME=/opt/upmem-2023.2.0
```

You should also have a recent `C/C++` compiler installed on your system that supports at least C++20.

1.2.2 Getting the *BPL*

Currently, the simplest way to obtain the *BPL* is to clone the following Git repository:

```
https://github.com/BioPIM/BPL.git
```

Once the repository has been cloned, you can do the following to build the unit tests from the cloned directory:

- mkdir build
- cd build
- cmake ..
- make

During the build, you should see a message instructing you to set the `DPU_BINARIES_DIR` environment variable to a specific value. This is mandatory in order to run the unit tests, because the *BPL* needs to know where the binaries generated for the *UPMEM* architecture are located (more on this later).

The unit tests[3] can be launched from the build directory as follows:

```
./test/unit/host/bpl-unittests.host
```

1.2.3 Creating a project from scratch

It is possible to write a new program from this content, but this repository is primarily focused on the development of the library itself. A useful way to create an independent *BPL* project is to generate a `tar.gz` archive containing the minimal source code and CMake files required to build a *BPL* project that can run on both multicore and *UPMEM* architectures.

To do this, you can use CMake while specifying the name of your project. For example, if you want to name your project MyProject, you would do the following:

[3]The unit tests are based on the Catch2 framework

```
cmake -DNEWPROJECT_NAME=MyProject ..
```

This will add a specific target, `create_bpl_project_MyProject`, and the project can then be generated with:

```
make create_bpl_project_MyProject
```

This will produce a `MyProject.tar.gz` archive containing the minimal resources for your project. You can then do:

1. `tar xvfz MyProject.tar.gz`
2. `cd MyProject`
3. `mkdir build`
4. `cd build`
5. `cmake ..`
6. `make`

Don't forget to set the `DPU_BINARIES_DIR` environment variable as indicated in the build output. You can then run the program with:

```
./src/main/MyProject
```

1.2.4 Compilation toolchain with the *BPL*

The *BPL* aims to make the code agnostic with respect to the underlying architecture. This can be quite challenging for architectures like *UPMEM* . Normally, a developer would have to write two programs: one for the host using standard C/C++, and one for the DPU chipset using the *UPMEM* SDK. With the *BPL* , the developer does not need to worry about these details and can focus on writing a single codebase. To achieve this, the *BPL* provides a compilation toolchain based on *CMake*, which handles the generation of the two binaries (one for the host and one for the DPU) whenever the *UPMEM* architecture is targeted.

This compilation toolchain requires that the developer follow certain conventions:

1. The task is implemented as a functor, i.e., a structure that defines an operator() method performing the computation.
2. This structure is templated, with a ARCH template parameter representing the target hardware architecture.
3. The structure must be implemented in a .hpp file named after the structure; that is, if the structure is named X, the header file must be named X.hpp and placed in the src/tasks directory.
4. The task's operator() method can take input parameters; these parameters are the only data the task can process.
5. The task is invoked through the bpl::Launcher class and its run method.

6. The launcher can be defined, for instance, in the program's main function or in any source file located in the src/main directory.

7. The bpl::Launcher class takes the target hardware architecture as a template parameter (e.g., multicore, *UPMEM* , etc.); additional arguments can be provided to the constructor to define the resources to use (number of threads, tasklets, etc.).

8. Normally, the only place where a hardware architecture is explicitly referenced is when instantiating the bpl::Launcher class; all other code should not rely on architecture-specific details.

9. When N process units are used, the bpl::split function can be applied to one or more input arguments, splitting each argument so that each process unit receives a specific portion as input.

In brief, the toolchain assumes that task-related code is placed in the src/tasks directory, and launcher-related code for running one or more tasks is placed in the src/main directory. Based on this structure, the *BPL* can generate the required binaries: one for the multicore architecture, and one plus N (where N is the number of tasks) when targeting the *UPMEM* architecture.

1.2.5 Implementing a task with the *BPL*

As noted earlier, a task is a templated structure that looks like

```
template<typename ARCH>
struct Sum
{
    USING(ARCH);
    int32_t operator() (vector<int32_t> const& in) const
    {
        int32_t result = 0 ;
        for (auto n : in)  { result += n; }
        return result ;
    }
} ;
```

At first glance, it looks like standard C++, but a closer look raises a question: what is the actual class for vector? We are not referring here to the std::vector class (assuming we have not used using namespace std earlier), so the compiler would report an error indicating that vector is unknown.

A type such as std::vector normally relies on dynamic allocation (i.e., new/delete). This is not possible on the *UPMEM* architecture, so if we want a class that behaves like std::vector, we must provide a specific implementation from another namespace rather than std. In fact, the actual vector class should depend on the template parameter ARCH, so one could write something like

```
template<typename ARCH>
struct Sum
{
```

```
    int32_t operator() (ARCH::vector<int32_t> const& in) const
    { /* same as before */ }
} ;
```

This will not compile, and the correct type for in should instead be:

```
typename ARCH::vector<int32_t> const&
```

This is somewhat verbose. To avoid this, one can use the USING macro, which acts like a using directive for a set of predefined types. With the USING macro, it becomes possible to skip prefixing each type (such as vector) with ARCH [4], and we can write something like

```
template<typename ARCH>
struct Sum
{
    USING(ARCH);
    int32_t operator() (vector<int32_t> const& in) const
    { /* same as before */ }
};
```

In this example, std::vector will be used when targeting a multicore architecture, while bpl::vector will be used for the *UPMEM* architecture. Here, bpl::vector is a specific implementation that handles the lack of dynamic allocation on *UPMEM* . Note, however, that bpl::vector implements only a subset of the methods available in std::vector. Therefore, one should rely only on this subset to ensure that the task code remains compatible with both multicore and *UPMEM* architectures.

This remark highlights an important caveat for the *BPL* : the goal is to have a single codebase that works across different architectures, but this comes at a cost. While it is convenient to use data structures like vector in a uniform way, it is not always possible to maintain identical semantics between multicore and *UPMEM* implementations. For example, methods in std::vector that rely on exceptions cannot be used on *UPMEM* , as exceptions are not supported there. As a result, functional differences may occur when running such methods on multicore versus *UPMEM* architectures.

[4] Technically, we are not using namespaces here as in using namespace, but rather defining type aliases.

Chapter 2

The design of the BPL

2.1 Introduction

The BPL has been designed with the goal of providing a programming model very close to a standard C++ approach, notably through the use of the standard library. The underlying idea has always remained the same, namely the ability to write a single piece of code that can run on different hardware architectures; many design choices were driven by this objective.

The BPL design process proceeded as follows: numerous user snippets were written to get an idea of what developers would like to be able to write. Analyzing these snippets allowed us to distinguish what was feasible from what was not, given the constraints of the UPMEM architecture (data broadcast, tasklet model, etc.), notably by rewriting the equivalent of these snippets using the UPMEM SDK.

It quickly became apparent that it was possible to achieve a programming model close to a "standard" C++ style, by hiding low-level UPMEM SDK calls in various parts of the library.

In this context, the C++ language has the important capability of "modifying" source code according to certain statically known criteria, allowing the generation of source code that will ultimately be compiled into an executable. This capability is generally referred to as metaprogramming and it was widely used during the implementation of the BPL.

In addition to this document, interested readers are invited to consult documents BPL_PreliminaryIdeas.odp and BPL_ShortPresentation.odp which detail the ideas developed during the initial design of the BPL and that ultimately led to its current implementation.

2.2 Task, Launcher and architectures

From the developer's point of view, Task and Launcher are the two most visible classes in the entire *BPL* API, which explains why particular care was taken in defining their design.

For example, it was important that the number of references to the hardware architecture to be used be as small as possible, with the goal of being able to switch easily from one architecture to another. The most favorable option was to localize this choice in a single place and, naturally, this is linked to the entity responsible for executing a task, namely the class bpl::Launcher. Thus, the Launcher class is the only place where information about the hardware architecture to be used appears. For example:

```
Launcher<ArchUpmem> launcher;
```

or

```
Launcher<ArchMulticore> launcher;
```

Note that the architecture is parameterized via a template parameter; in other words, the Launcher class is actually a template class whose template parameter ARCH represents a given architecture. From an implementation point of view, the Launcher class has an attribute arch_ of type ARCH, and most of the functionality of Launcher is forwarded to the ARCH class through the arch_ attribute.

One may note that it would have been possible to configure a launcher's architecture as a constructor parameter, for example:

```
Launcher launcher (ArchUpmem{});
```

However, this choice was not retained for several reasons:

- The Launcher class would have had to know about all possible architectures, for example through the use of a std::variant. Adding a new architecture would therefore have required a modification of the Launcher class. By using a template parameter, a new architecture can be added without having to modify Launcher.
- This choice would have enabled a more dynamic approach, namely the possibility of changing the type of architecture of a launcher at runtime. However, dynamic binding often implies an overhead that is unnecessary in this specific case: indeed, it is very unlikely that one would need to change a launcher's architecture on the fly.
- Even if it may seem restrictive, it appears more natural to associate a launcher with an architecture at the type level. If there is a need to manage two architectures, it is sufficient to create two launchers, each with a different template.

Once the architecture is associated with a launcher, we have a type for which certain arguments can be provided to the constructor, knowing that the constructor will therefore be tied to the underlying architecture. Thus, if the architecture is statically bound to the launcher, the configuration of the architecture is done through the launcher's constructor, for example:

```
Launcher<ArchUpmem> launcher (4_dpu);
```

or

```
Launcher<ArchMulticore> launcher (16_thread);
```

The constructor may take different parameters depending on the type of architecture; for more details, one can refer to the corresponding API.

Once a launcher has been instantiated, it becomes possible to execute a task via the run function. Here again, two possibilities were conceivable for launching the execution:

```
launcher.run (Checksum{}, std::vector {1,2,3,5,8});
```

or

```
launcher.run<Checksum> (std::vector {1,2,3,5,8});
```

However, the second option is the only feasible one because the task (as we will see shortly) is itself a template class whose parameter is the architecture used by the launcher. Thus, the first option would have required writing something like this:

```
Launcher<ArchUpmem> launcher (4_dpu);
launcher.run (Checksum<ArchUpmem>{}, std::vector {1,2,3,5,8});
```

which involves some redundancy, with the architecture appearing in two different places - something we want to avoid.

With the second option, we can write this without redundancy:

```
Launcher<ArchUpmem> launcher (4_dpu);
launcher.run<Checksum> (std::vector {1,2,3,5,8});
```

because the launcher is aware of the architecture and can therefore pass the corresponding type as a parameter to the task when it is time to instantiate it.

Moreover, this second option allows for a clear separation between the task (template argument) and the input arguments to be provided to the task (arguments of the run function), which makes the code easier to read.

Providing the task as a template parameter of the run function has an additional advantage: it is also possible to provide, after the task type, additional template parameters that can be used to further configure the task. Here is an example:

```
template<class ARCH, typename...TRAITS>
struct TemplateTask {
    using T = std::tuple_element_t<0,std::tuple<TRAITS...>>;
    auto operator() (const T& arg)  {  return arg;  }
};
```

We can see how TemplateTask accepts, in addition to the architecture, other types that can be used in the implementation of the task. It can be called as follows:

```
double value {145};
Launcher<ArchMulticore> launcher {4_thread};
auto res = launcher.template run<TemplateTask,decltype(value)> (value));
```

However, this poses a problem when using the UPMEM architecture. Indeed, the additional type to be provided to the task (here a double) is only known at the call site, that is, from the host. This call site is not known in the DPU source code, which is by nature generic (see the section on this topic).

We will therefore need to work around this by indicating at the CMake level that it is indeed a TemplateTask <ARCH,double> that we are interested in; by convention,

we must explicitly add this specialization, TemplateTask <double>[1], to the TASKS_LIST variable. If another specialization is desired, for example TemplateTask <ARCH,int>, a second version, TemplateTask <int>, must be added to the CMake variable TASKS_LIST. If a specialization is used on the host side that has not been anticipated in CMake, an error will occur.

So far, we have outlined the main choices that guided the design of the Launcher class; we will soon look at some specifics for the ArchUpmem and ArchMulticore architectures. Before that, let us return in more detail to the decisions that determined the design of a class implementing a task.

As we have already seen roughly before, a class implementing a task executable by a launcher must follow certain conventions.

First of all, it must be a template class with at least one template parameter corresponding to the architecture.

```
template<class ARCH> struct MyTask {};
```

Next, the function that performs the expected task is an overload of the operator(). Historically, an earlier version of the BPL rather offered a static method run, e.g.

```
template<class ARCH> struct MyTask {
   static auto run (int n) { return 2*n; }
};
```

This solution was ultimately abandoned in order to better conform to the concept of function objects (see https://en.cppreference.com/w/cpp/functional.html), which has the advantage of being closer to the spirit of standard C++ and potentially reusable via std algorithms (something not possible with the static run API). We therefore ultimately opted for:

```
template<class ARCH> struct MyTask {
   auto operator() (int n) const { return 2*n; }
};
```

The definition of a task must be located in a header file. By convention, if the class implementing the task is named X, the header file containing its definition must be named X.hpp. This convention is necessary for the CMake toolchain.

We will now detail the mechanism that allows the BPL to "naturally" write code that can be executed on a UPMEM architecture. One of the main drawbacks of the UPMEM model is that the developer must manually handle both the serialization of the task's input arguments and their broadcast to the DPUs. In fact, if both the host source code and the DPU source code share knowledge of the signature of the task to be executed on the DPU, a major step toward simplifying the programming model is achieved. Indeed, knowing the task's prototype allows the use of a dedicated serialization framework (see the chapter on serialization):

- On the host side, the task definition must be known in order to know the prototype of the operator(); when a call such as launcher.run<MyTask>(4) is made, knowing

[1] note that we don't mention ARCH here, this is just a convention used by the configuration_file command of CMake

this prototype allows the argument 4 to be interpreted as an integer that can be serialized into a buffer to be broadcast to a DPU. Note that the host only uses the prototype of operator() and not its implementation, which is normal because the operator must be executed on the DPU, not on the host.

- On the DPU side, the task definition must also be known to understand the prototype of operator() in order to deserialize the input arguments. But the implementation of operator() must also be known, because this is the function that will actually be executed.

Thus, the task's source code must be known both to the host and to the DPU, but for different reasons. This implies that both the host and DPU source code must #include the header file containing the task's definition. Incidentally, the need for static analysis of the operator() prototype further highlights the particular usefulness of C++ and its metaprogramming capabilities.

2.2.1 ArchUpmem

We will now look at some additional aspects related to the ArchUpmem class. This is the class provided as a template parameter to the Launcher class when the UPMEM architecture is targeted, and it performs the majority of actions by delegation from the launcher.

For instance, let us analyze what is involved on the host side for a call such as:

```
auto result = Launcher<ArchUpmem>{1_dpu}.run<MyTask>(4);
```

Since we are targeting a UPMEM architecture, the launcher must perform the following actions through the class ArchUpmem:

- Analyze the prototype of MyTask::operator() in order to serialize the arguments (here, the integer 4).
- Once the serialization buffer is configured, it is broadcast to the DPUs via the UPMEM SDK's "scatter/gather" mechanism. If one of the arguments is wrapped using the split function, additional actions must be taken.
- The DPU binary corresponding to the task is loaded into the various DPUs. This, of course, assumes that the associated DPU binary has been previously compiled by the CMake toolchain.
- The DPU binary is launched for each tasklet on each DPU.
- When execution is complete, the host retrieves the serialization buffer corresponding to the task's return via the UPMEM SDK.
- This buffer is deserialized to produce a vector of N entries, each entry being the result emitted by each tasklet. Note that the element type of this result vector is the same as the return type of MyTask::operator(), here an int.

In the end, it turns out that the Launcher::run function handles most of the low-level calls to the UPMEM SDK, making it possible for the developer to use a "natural"

programming style, closer to a simple function call. We can see particularly in this simple example how it is possible to express in a single line what would require many lines when using the UPMEM SDK directly.

Regarding the retrieval of tasklet results on the host side, it should be noted that there are currently two implementations. Historically, the first implementation was generic, relying on the Serializer::from function. However, when a tasklet returned a vector, the mechanism of serializing the result on the DPU side and then deserializing it on the host side could be relatively costly in terms of both memory and execution time.

There is now an optimized implementation specifically for the case where the task's return type is a vector; this provides a significant performance gain and reduces *BPL* overhead in situations where a direct approach via the UPMEM SDK might have been more efficient. For reference, see the different template specializations of the result_wrapper class in the ArchUpmem.hpp file.

There are many other interesting aspects regarding the ArchUpmem class, but these fall more into the realm of implementation details than design concepts. Interested readers can consult the source code of this class for further information, particularly through the comments.

2.2.2 ArchMulticore

The ArchMulticore class allows instantiating the Launcher class to use a thread-based parallelization model on a multicore architecture.

It should be understood that a task executed via such a launcher will actually run on the host side. As a result, the implementation of ArchMulticore::run is much simpler than that of ArchUpmem::run and essentially resembles traditional thread-based programming.

There is therefore no concept of information broadcasting, nor of serialization and deserialization, since the objects are already present in the host memory. Note the use of a thread pool here, which helps minimize the creation and destruction of threads, operations that can be costly from the operating system's perspective. For reference, the BS::thread_pool library is used here.

One particular aspect concerns the management of the number of threads used. Indeed, note the presence of the following constructor (simplified prototype for clarity):

```
ArchMulticore (TASKUNIT taskunit, TASKUNIT chunksize);
```

The first argument represents the number of process units (i.e., threads in this context) available to the launcher, while the second represents the maximum number of process units that can be used at any given time. For example:

```
Launcher<ArchMulticore> (1024_thread, 16_thread);
```

In this example, the launcher will virtually have 1024 threads but will only be able to execute 16 physically at a time. This is useful because the taskunit parameter (1024 in the example) is the value used to split an argument when it is wrapped by the bpl::split function. This is convenient in cases where one wants the same level of splitting as a rank in UPMEM, knowing that a rank has 1024 process units (i.e., tasklets in this context).

2.3 The "split" process in the BPL

We briefly saw earlier the parallelization model proposed by the *BPL* . The task has no knowledge of any parallelization scheme; it is the responsibility of the caller (i.e., the host) to split the input data so that each processing unit gets a specific portion of that data. In other words, a partition of the input data is created, and each element of this partition is processed by a processing unit. This gives the developer the freedom to decide what to parallelize; one can thus partition one or several task arguments.

The *BPL* provides a simple way to partition an argument when launching a task; it is enough to wrap this argument with the bpl::split function, for example:

```
auto results = Launcher<ArchUpmem>{8_dpu}.run<Checksum>(split(somevector));
```

Technically, the return type of the split function is not the same as that of its input argument (a vector of integers in the example); it is of type SplitProxy, which is a class template. Here is an excerpt:

```
template<typename L, bpl::SplitKind KIND, typename TYPE>
struct SplitProxy  {
    using arch_t = typename L::arch_t;
    SplitProxy (const type& t) : _t(t)  {}
    operator const type& () const { return _t; }
    const type& _t;
};
```

Essentially, this class holds a reference to the object provided at construction, but it also contains a number of type-related pieces of information, notably information regarding the desired architecture as well as the level of parallelism - a point that we will detail very soon.

At first glance, it may seem surprising that one can store this type information in addition to the reference to the encapsulated object. However, we can see that the call to split occurs when passing arguments to the Launcher::run function, which means that in this context, the architecture is known, since it is the template parameter of the Launcher class.

Important point: the SplitProxy class defines a conversion operator that allows it to return the encapsulated object, which will be necessary when one actually wants to use the object for task execution. What should be retained is that the split function acts as a tag for the encapsulated object, associating it with type information needed for partitioning the object.

Now, let us return to the different levels of parallelism depending on the architecture.

- in the case of a multicore architecture, the scheme is simple because there is only a single level of parallelism, namely N threads executing simultaneously.

- in the case of the UPMEM architecture, there are three levels of parallelism: the rank, consisting of 64 DPUs, with each DPU executing 16 tasklets. Additionally, there are certain synchronization constraints: a rank must wait for all 64 of its DPUs to finish execution, and a DPU must wait for all 16 of its tasklets to complete.

Thus, the implementation of the split function must be able to handle different levels of parallelism if necessary. This information is stored in the bpl::SplitKind parameter of the SplitProxy class. By default, each architecture defines a default level of parallelism; for multicore, this will be:

```
using lowest_level_t = Thread
```

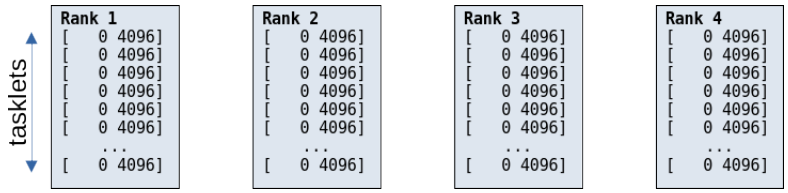
and for UPMEM

```
using lowest_level_t = Tasklet;
```

To make things more concrete, consider the following example:

```
// some integer range
pair<int,int> range (0, 4096);
// our launcher
Launcher<ArchUpmem> launcher {4_rank};
// launch the execution of some task
launcher.run<DoSomething> (range);
```

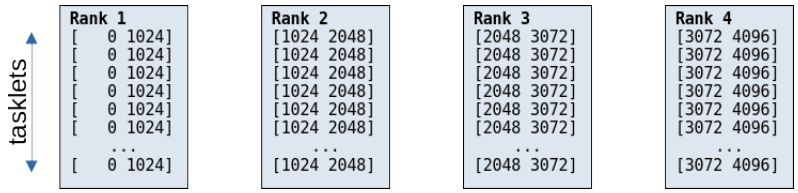
Here, the split function does not encapsulate the range argument, so the tasklets of each DPU in each rank will see the same object like this (here, we assume that we know how to partition an interval of integers):



Now, we can partition by rank by using split with the corresponding constant: each tasklet of the same rank will receive the same data:

```
launcher.run<DoSomething> (split<ArchUpmem::Rank>(range));
```

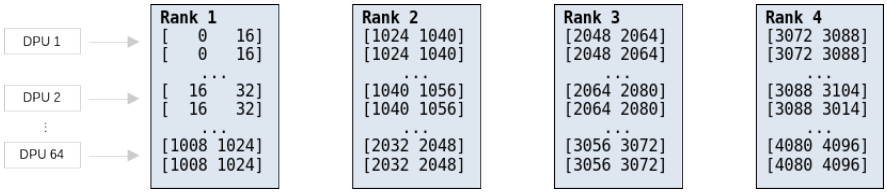
The data will be partitioned as follows:



We can partition by DPU by using split with the corresponding constant: each tasklet of the same DPU will receive the same data:

```
launcher.run<DoSomething> (split<ArchUpmem::DPU>(range));
```

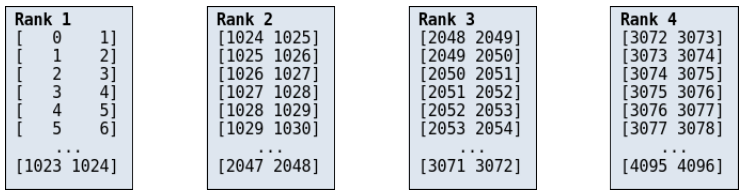
The data will be partitioned as follows:



We can partition by tasklet by using split with the corresponding constant (or without specifying it since this is the default value): each tasklet will receive specific data:

```
launcher.run<DoSomething> (split<ArchUpmem::Tasklet>(range));
```

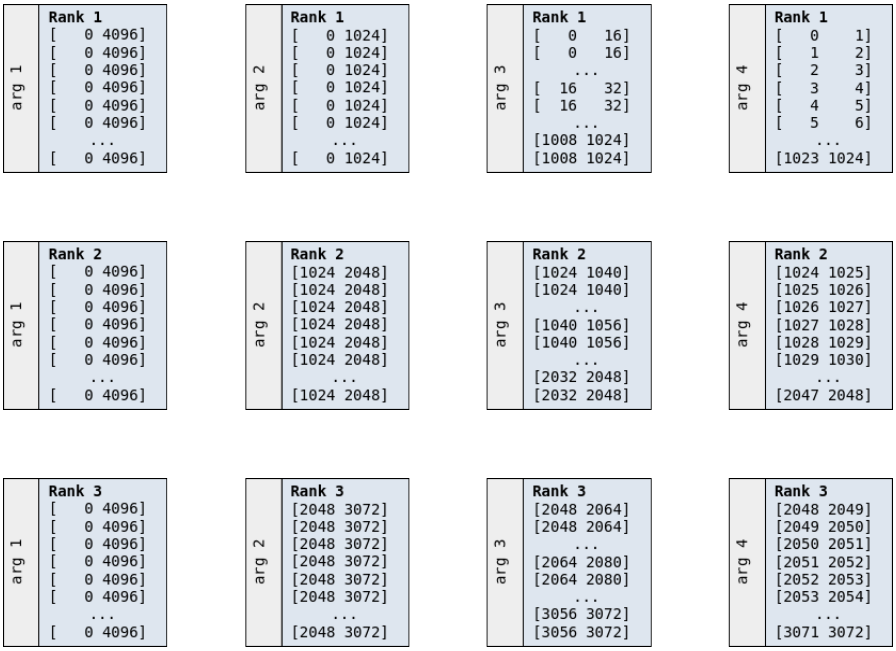
The data will be partitioned as follows:

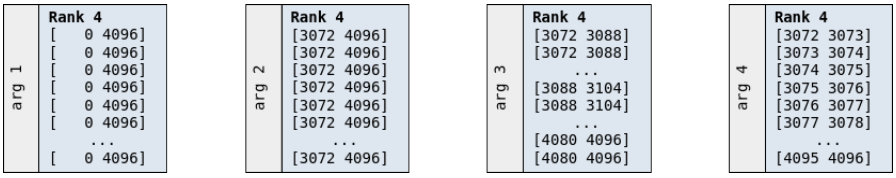


It is possible to combine different levels of partitioning for different arguments.

```
pair<int,int> range (0, 4096);
Launcher<ArchUpmem>{4_rank}.run<DoSomethingElse> (
  range,                                  // arg 1
  split<ArchUpmem::Rank>    (range),      // arg 2
  split<ArchUpmem::DPU>     (range),      // arg 3
  split<ArchUpmem::Tasklet> (range)       // arg 4
);
```

which will result in the following partitionings for the 4 ranks:





Now, let's detail how a partition can be created from an object. As we saw, a call to the split function will return a SplitProxy object containing a reference to the encapsulated object as well as type information.

We define now a class SplitOperator intended to be specialized, particularly with a 'split' method that defines how to split a type T. Such specializations are provided for types such as a pair of numerics, e.g.:

```
template <typename A,typename B>
struct SplitOperator<std::pair<A,B>>  {
    static auto split (
        const std::pair<A,B>& t,
        std::size_t idx,
        std::size_t total
        ) {
        size_t diff = t.second - t.first;
        size_t i0   = diff * (idx+0) / total;
        size_t i1   = diff * (idx+1) / total;
        return std::pair { t.first+i0, t.first+i1 };
    }
};
```

The idea is as follows: we provide as input the object to partition, along with the index of the partition to be generated and the total number of elements in the partition. In this specific case, we divide the input interval into several sub-intervals. It is worth noting that the result type of this split function is the same as the specialized type.

Now for std::vector:

```
template <typename T>
struct SplitOperator<std::vector<T>>  {
    static auto split (
        const std::vector<T>& t,
        std::size_t idx,
        std::size_t total
     ) {
        auto range    = std::make_pair (0, t.size());
        auto subrange = SplitOperator<decltype(range)>::split (
            range, idx, total);
        return std::vector<T> (
            t.begin()+subrange.first,
            t.begin()+subrange.second);
    }
};
```

It is worth noting that each element of the partition is itself a vector, which is a copy of

a subpart of the input vector. This copy can, in some cases, be unnecessary and costly in terms of memory, so it is possible to write a second function called split_view, which helps avoid data copies by using std::span.

```
template <typename T>
struct SplitOperator<std::vector<T>>  {
    static auto split (
        const std::vector<T>& t,
        std::size_t idx,
        std::size_t total
     ) {
        auto range    = std::make_pair (0, t.size());
        auto subrange = SplitOperator<decltype(range)>::split (
            range, idx, total);
        return std::vector<T> (
            t.begin()+subrange.first,
            t.begin()+subrange.second);
    }
    static auto split_view (
        const std::vector<T>& t,
        std::size_t idx,
        std::size_t total
    ) {
         return SplitOperator<std::span<T>>::split (
            static_cast<std::vector<T>&>(t), idx, total);
    }
};
```

the specialization for std::span being:

```
template <typename T>
struct SplitOperator<std::span<T>> {
    static auto split (
        std::span<T> t,
        std::size_t idx,
        std::size_t total
    ) {
        auto range    = std::make_pair (size_t(0), t.size());
        auto subrange = SplitOperator<decltype(range)>::split (
            range, idx, total);
        auto res = t.subspan (
            subrange.first, subrange.second - subrange.first);
        return res;
    }
    static auto split_view (
        std::span<T> t,
        std::size_t idx,
        std::size_t total
    ) {  return split (t, idx, total);  }
};
```

To be exhaustive, we will note the specialization for the SplitProxy type, which simply forwards to the type of the encapsulated object.

```
template<typename LEVEL, bpl::SplitKind KIND, typename TYPE>
struct SplitOperator<details::SplitProxy<LEVEL,KIND,TYPE>> {
    static decltype(auto) split (
        const details::SplitProxy<LEVEL,KIND,TYPE>& t,
        std::size_t idx,
        std::size_t total
    ) {
        return SplitOperator<TYPE>::split (t, idx, total);
    }
};
```

This mechanism can be extended to a user-defined type; in that case, a template specialization of the SplitOperator class should be written for this user type.

2.4 Reducing the results

Parallelizing an algorithm often involves obtaining several partial results that must be aggregated at the end of the computation. For example, one can compute the sum of the elements of a vector as follows:

```
auto results = Launcher<ArchUpmem>{8_dpu}.run<Checksum>(split(somevector));
```

We note that:

- a launcher with 8 DPUs is used, meaning that there will be $8 \times 16 = 128$ processing units (tasklets) available
- the vector passed as an argument is wrapped by the bpl::split function. This results in the vector being split into 8 distinct parts, and then each DPU further splits its vector into 16 parts, one specific part for each tasklet
- the result stored in the results variable will therefore be a vector of 128 integers, each integer representing a partial checksum of the original vector.

Technically, the type of the variable result is not necessarily std::vector<int64_t >; certain optimizations may take place and therefore return something other than a std::vector. However, it is guaranteed that the return type of Launcher::run is something iterable, so in all cases it will be possible to reduce the result to a final integer, equal to the sum of all partial checksums.

```
int64_t sum = 0;
for (auto x : results)  { sum += x; }
```

Reducing the result to a single final object is a fairly common operation, so the *BPL* provides a mechanism to automate this process; the developer can implement, at the task level, a reduce method that performs the reduction:

```
template<class ARCH>
struct Checksum : bpl::Task<ARCH> {
    USING(ARCH);
    auto operator() (vector<uint32_t> const& v)   {
        uint64_t checksum = 0;
        for (auto x : v)  {  checksum += x;  }
        return checksum;
    }
    static uint64_t reduce (uint64_t a, uint64_t b)  { return a+b; }
};
```

The following points should be noted:

- the reduce method must be declared static
- its signature must be consistent with the return type of operator()

Important point: the presence of a reduction method at the task level will change the return type of Launcher::run. Instead of an iterable object over all partial results, it will return a single object (a uint64_t in the example), so one can write directly:

```
uint64_t res = Launcher<ArchUpmem>{8_dpu}.run<Checksum>(split(somevector));
```

Another remark: the final reduction is performed on the host side, which means that all partial results must first be broadcast from the DPUs to the host (i.e., 1024 partial results in the Checksum example). An optimization could be achieved by performing a partial reduction on each DPU, which would then reduce the 16 partial results (coming from the 16 tasklets) into a single one. This would have the advantage of increasing the degree of parallelism (i.e., making better use of the DPUs) and reducing the amount of information broadcast between the DPUs and the host.

From a technical point of view, the reduction is performed at the level of the Launcher class, not at the level of an architecture class such as ArchMulticore or ArchUpmem. A type trait has_reduce is used to detect the presence or absence of a static reduction function at the task level.

```
template <typename T>
class has_reduce {
    typedef char one;
    struct two { char x[2]; };
    template <typename C> static one test( decltype(C::reduce) ) ;
    template <typename C> static two test(...);
public:
    enum { value = sizeof(test<T>(0)) == sizeof(char) };
};
```

Next, a template structure Reduce is defined, which uses template specialization to provide a version that either performs a reduction or not, depending on whether the result is received in an iterable form.

```
// Generic definition
template<bool,class TASK>  struct Reduce;

// Specialization 1: reduce the partial results by using 'reduce' method
template<class TASK>  struct Reduce<true,TASK> {
    using Result_t = return_t <decltype(&TASK::operator())>;
    template<typename RESULT_RANGE>
    auto operator() (const RESULT_RANGE& results) const {
        Result_t res = Result_t();
        for (auto&& result: results) { res = TASK::reduce(res,result); }
        return res;
    }
};

// Specialization 2: return the partial results
template<class TASK>  struct Reduce<false,TASK> {
    template<typename RESULT_RANGE>
    decltype(auto) operator() (RESULT_RANGE&& results)  {
        return std::move(results);
    }
};
```

This example of using template specialization is very characteristic of the way the *BPL* uses metaprogramming techniques. Many similar examples can be found throughout the implementation of the *BPL* .

2.5 Serialization

One of the crucial aspects of UPMEM programming is the need to transfer the data to be processed from the host to the DPUs, and conversely to retrieve the computation results from the DPUs back to the host. From this point of view, this is very close to a data transfer model over a network. This has two implications:

- the data to be transferred can be of very different types (scalars, vectors, etc.), potentially including user-defined types. In order to be able to carry this information, it must first be serialized, i.e., converted into a generic format. The result is a byte buffer that can be transported over a network, or in the case of UPMEM, between the host and the DPUs.
- once the data have been serialized into a buffer, the UPMEM SDK must then be used to perform a host/DPU or DPU/host broadcast. There are several ways, with varying efficiency, to perform these transfers. A naïve approach assumes a single buffer to be broadcast from the host to the DPUs, for example. If the data to be broadcast on the host side are not contiguous in memory, this requires copying all the data beforehand in order to obtain a contiguous buffer in memory, which implies an expensive copy both in terms of memory space and execution time. However, the UPMEM SDK provides a mechanism called scatter/gather that allows broadcasting non-contiguous regions in host memory, and this is therefore the option favored by the BPL, albeit with increased implementation complexity.

Another point concerns the need to handle the "split" of certain arguments provided as input to the task at the serialization level. This additional requirement must be managed on top of the use of scatter/gather.

Ultimately, all these specificities make it difficult to use an existing serialization framework. For example, it had been considered to use the "cereal" framework (more details can be found at https://uscilab.github.io/cereal), but adapting it to the context of the BPL proved challenging, which led to the development of a custom implementation for the serialization part.

From an implementation point of view, serialization is handled by the Serialize class. This is a template class whose template parameters are:

- class ARCH: the architecture used, defined when instantiating the launcher
- class BUFITER: the buffer type used to scan a buffer containing the information to be deserialized
- int ROUNDUP: an integer used to round certain values when specific memory alignment constraints must be respected (e.g., memory alignment to 8 bytes)

In a generic way, the Serialize class provides two methods: iterate, which allows serializing an object into a buffer, and restore, which allows initializing an object from a serialized buffer. Serialization for different types uses template specialization of the Serialize class. Thus, a given type can provide its own specialization in order to become serializable.

The serialize.hpp file therefore provides template specializations for a number of commonly used types (scalars, vectors, arrays, tuples, PODs, etc.). The developer has still the possibility to provide their own specialization for a specific type.

Historically, the SFINAE approach was used to implement the various specializations. By moving to the C++20 standard, it became possible to use the concept feature, which makes the code lighter and more understandable.

For reference, we will illustrate here two cases of serialization:

- vector: Since the number of elements is only known at runtime, we must serialize (a) the number of elements in the vector and (b) the buffer containing the vector's data.
- array: Unlike the vector, the number of elements is known statically, so it is sufficient to serialize the buffer containing the array's data.

It should be noted that the situation is a bit more subtle: it is only possible to serialize the internal data buffer directly if the type of the data is POD (Plain Old Data).

As mentioned previously, the handling of split and scatter/gather makes the implementation relatively complex. The reader can refer directly to the implementation in the serialize.hpp file to understand the finer details, as well as the implementation of other type specializations.

2.6 DPU source code

One of the challenges of the BPL for the UPMEM architecture is to allow the developer not to deal with the low-level aspects of the UPMEM SDK. In addition, it is also important

to provide a programming model that is as standard as possible, such as the use of the C++ standard library.

A delicate aspect of UPMEM programming is that it is necessary to develop not one but two programs: one for the host side and the other for the DPU side. From the BPL's perspective, and in order to keep the programming model as generic as possible, it is important to smooth out this specificity and behave as if only a single program had to be developed.

One possible approach is to ensure that there is a single generic source code for the DPU binary. Since this code must in practice execute the desired task, this generic code must be able to include the source code of the task. The constraint is therefore as follows: the source code of the DPU binary is generic, with an inclusion of the header file containing the source code of the task.

One can take advantage of the CMake tool and its ability to configure a generic file by adapting it according to one or more variables. In the following, we will refer to ArchUpem.dpu.A.cpp.in as the name of the generic file for the DPU source code. To fix ideas, here is an excerpt from this file (this is not exactly the real file, but it conveys the overall idea):

```
#include <bpl/core/Task.hpp>
#include <bpl/arch/ArchUpmemResources.hpp>
#include <tasks/@TASKNAME@.hpp>

static const std::size_t ARGS_SIZE = 62*1024*1024;

// This is the main buffer holding the information from the host.
__mram_noinit  uint8_t __args__ [ARGS_SIZE];

int main()
{
   // we define the type of the task
   using task_t = @TASKNAME@ <bpl::ArchUpmemResources<>>;

   // we instantiate a task object
   task_t task;

   // we retrieve the actual arguments from the serialized buffer
   // coming from the host
   auto args = unserialize (__args__);

   // we execute the task with the incoming arguments
   auto result = task (std::forward<decltype(args)>(args)...);

   // we prepare the result for DPU->host broadcast
   serialize (result);
}
```

From the point of view of the CMakeLists.txt file, we will have:

```
configure_file(
  ${PROJECT_SOURCE_DIR}/src/bpl/arch/dpu/ArchUpmem.dpu.A.cpp.in
  ${OUTNAME}.cpp
  @ONLY
)
```

which will generate the final .cpp file for the source code of the DPU binary. The configure_file command replaces the @TASKNAME@ variable with the name of an actual task (e.g., Checksum), which results in the inclusion of the Checksum.hpp file. As a consequence, this specialization of the generic DPU source file is aware of the definition of the Checksum task and is able to execute it.

This is the general scheme for building the DPU binary. There are a number of implementation details that will not be discussed here, but which can be listed for reference:

- computation of certain constants at compile time (e.g., vector cache sizes)
- handling of the MetadataInput metadata coming from the host; these metadata include, for example, information required for deserialization (e.g., the once tag);
- partitioning of arguments into two categories depending on the presence of the global tag; by default, arguments are declared on the execution stack, i.e., inside the main function. Arguments tagged with global are declared outside the main function and are therefore accessible by the different tasklets;
- handling of arguments encapsulated by split; that is, an argument can be "split" into different parts, each part being specific to a tasklet;
- handling of the MetadataOutput metadata, which provide the host with the information required for deserialization. These metadata also include statistical information that can be used by an expert to study potential overheads related to the BPL.
- for certain return types, post-processing is required; for example, a vector may potentially be in a dirty state, meaning that the internal cache has not been flushed to MRAM, and therefore the vector must be flushed to MRAM before preparing serialization to the host.
- preparation of the broadcast of the result to the host

2.7 Tagging classes for behaviour customization

2.7.1 Why tags?

The philosophy of the *BPL* is to write a single code and be able to execute it on different hardware architectures via the use of a launcher. However, it may happen that an architecture is specific enough that we need to 'assist' the *BPL* in addressing these specificities. In this section, these specificities will always be for the UPMEM architecture.

2.7.2 Tag once

A first example concerns a performance issue. Suppose we call the same task repeatedly, with a first argument that does not change from one call to another, and a second argument

that can vary:

```
auto r1 = launcher.run<DoSomething> (databaseReference, queries1);
auto r2 = launcher.run<DoSomething> (databaseReference, queries2);
auto r3 = launcher.run<DoSomething> (databaseReference, queries3);
```

with

```
template<class ARCH>
struct DoSomething  {
    USING(ARCH);
    auto operator() (
        vector<uint32_t> const& databaseRef,
        vector<uint32_t> const& queries)
    {
        // do something here
    }
};
```

In this example, both arguments will be broadcast from the host to the DPUs three times, even though the first argument does not change from one call to another. If this argument is large, this will result in two potentially unnecessary broadcasts that are costly in terms of execution time. It is worth noting that in the case of a multicore architecture, this poses no problem, since there is no broadcast to perform - the arguments are already present in the host memory.

It would therefore be useful to inform the *BPL* that the argument databaseReference only needs to be broadcast once, during the first call to Launcher::run. The *BPL* defines, in the form of a template class, a tag named once that will encapsulate the type of the argument that should only be broadcast once. The developer can use this tag directly at the task level as follows:

```
template<class ARCH>
struct DoSomething  {
    USING(ARCH);
    auto operator() (
        once<vector<uint32_t> const&> databaseRef,
        vector<uint32_t> const& queries)
    {
        // do something here
    }
};
```

The presence of this once tag will therefore provide the *BPL* with the indication to broadcast the first argument only during the first call to Launcher::run. Obviously, this tag will have no effect if the architecture used is ArchMulticore.

While being minimally invasive, this mechanism does require an adaptation in the use of the databaseRef object. Indeed, it can no longer be used directly as a vector since its type is no longer a vector. To access the underlying vector object, the once class overloads the operator*, so one can write for example:

```
auto operator() (
    once<vector<uint32_t> const&> databaseRef,
    vector<uint32_t> const& queries)
{
    for (auto r : *databaseRef) {  /* do something with r */  }
}
```

Logically, using once with an UPMEM architecture has a direct impact on the serialization mechanism. However, we will not go into the technical implementation details here.

2.7.3 Tag global

We previously saw how the *BPL* manages to use a single source code for the DPU binary. A first version of this code required that the arguments provided to the task be created in the call stack of each tasklet, with the consequence that a tasklet could not see the information used by other tasklets. However, sharing information between tasklets turned out to be a recurring need for developers, so an additional mechanism had to be implemented in the *BPL* .

Again, the idea is to tag a task argument to indicate to the *BPL* that the associated data can be shared among tasklets. This tag is implemented as a template class named global, which can be used in the following way:

```
template<class ARCH>
struct DoSomething  {
   USING(ARCH);
   auto operator() (global<array<uint32_t,128> const&> arg1, int arg2) {}
};
```

Here, the argument arg2 will typically be created in the call stack of each tasklet, which technically means the variable will be created in the tasklet's main function.

On the other hand, since arg1 needs to be shared, it must be created in a memory area other than the call stack[2]. In particular, this object must be declared as a global variable, outside of the main function. Therefore, we need to be able to reserve space in the WRAM that is separate from the space used for the call stacks of the different tasklets.

From the perspective of generic DPU source code, we therefore need to differentiate between objects created inside the main function and objects created outside it. Since, in the end, all arguments (whether tagged global or not) must be provided as arguments to the task to be executed, we create two tuples: one grouping the objects declared outside the main function and another grouping the objects declared inside the main function. The final tuple will then use references to the elements of these two tuples (taking care, of course, of any possible interleaving of global-tagged and non-tagged arguments).

Roughly speaking, the DPU source code looks like this (many details are omitted here, in particular, the way data coming from the host is deserialized into the two tuples):

[2]To remind you, the call stack occupies a certain portion of the WRAM

```
task_params_global_t globalParams;

int main()
{
    task_params_stack_t localParams;

    auto argsAsTuple = bpl::merge_tuples (globalParams, localParams);

    auto fct = [&] (auto &&... args) -> result_type  {
        return task_t{} (std::forward<decltype(args)>(args)...);
    };
    auto result = std::apply (fct, argsAsTuple);
}
```

The calculation of the sizes for the WRAM used by the call stacks and the WRAM used by the global-tagged objects is a key point handled by the CMake toolchain.

2.7.4 Tag glonce

The glonce tag is simply the conjunction of the global and once tags. Instead of using

```
auto operator() (global<once<vector<uint32_t> const&>> arg) {}
```

we can directly write

```
auto operator() (glonce<vector<uint32_t> const&> arg) {}
```

Using this tag will therefore allow you to benefit from both associated functionalities.

2.8 The genesis of the bpl::vector class

2.8.1 Introduction

In bioinformatics, it is common to work with data sequences, which are most often represented in C++ by vectors and/or arrays when the number of sequences is known a priori. The C++ standard, for example, provides the std::vector and std::array classes, which are widely used whenever one wants to work with data sequences.

However, when using non-standard hardware architectures like UPMEM, the availability of standard C++ libraries is limited. In the case of UPMEM, this unavailability stems from several factors:

- The UPMEM SDK is, by default, available in C. Although a C++ wrapper exists, it is very limited and does not allow the use of classes such as std::vector and std::array.
- A more fundamental reason exists for vectors. A classic implementation relies on the ability to allocate memory dynamically, but this capability is (almost) nonexistent in the UPMEM SDK. If one wishes to use vectors whose number of elements can vary dynamically, it is necessary to first implement a dynamic memory allocator on the DPU side.

This second point is particularly tricky because the working memory on the DPU is the WRAM, which is very limited in size - 64 KBytes that must be shared among 16 tasklets, leaving only 4 KBytes of memory available per execution stack. Consequently, it becomes necessary to use the larger-capacity MRAM to store data, with the WRAM serving at best as a temporary cache for a small portion of the complete structure, which is therefore stored in MRAM.

Using the WRAM as a cache for vector data is restrictive because the vector data of interest to the developer will not necessarily be in the cache, requiring a data transfer from MRAM to WRAM in order to access the desired information[3]. As a result, situations can arise where the number of cache invalidations (i.e., cache misses) becomes significant and penalizes overall performance. This leads to a scenario somewhat similar to a classical architecture with multiple levels of cache (L1, L2, L3) and main memory, where cache misses can occur at different cache levels.

To summarize, the outlook is not a priori very favorable for expecting, in the context of UPMEM, a library offering a class as useful as std::vector. However, at the cost of certain tradeoffs, it has been possible to design the *BPL* to allow writing code that is as close as possible to C++ code using the standard library. Here is an example of what can be written today with the *BPL* , where the same vector is iterated over using iterators to create a new vector by adding elements one by one:

```
template<class ARCH>
struct DoSomething  {
    USING(ARCH);
    auto operator() (vector<uint32_t> const& v) const {
        vector<pair<uint32_t,uint32_t>> result;
        for (auto it1=v.begin(); it1!=v.end(); ++it1)  {
            for (auto it2=it1+1; it2!=v.end(); ++it2)  {
                result.push_back (make_pair(*it1,*it2));
            }
        }
        return result;
    }
};
```

Before we begin, here are some features we would like to have in the API of this bpl::vector class:

- random access with operator[]
- iterators
- adding items with push_back

We will now discuss the design of the *BPL* -specific vector class, the API it provides, and its limitations.

[3]Recall that only data in WRAM is accessible to the tasklets

2.8.2 The bpl::vector class and its companion bpl::MemoryTree

We will assume that the vector contains elements of type T. We specify the bpl namespace here, but will omit it later for simplicity:

```
namespace bpl {
        template<class T> class vector {};
}
```

Since it is not possible to store the entire vector in MRAM due to the small size of this memory segment, the data must therefore necessarily be in MRAM. We can keep a portion of this vector in WRAM via a fixed-size array containing CACHE_NB_ITEMS elements of type T:

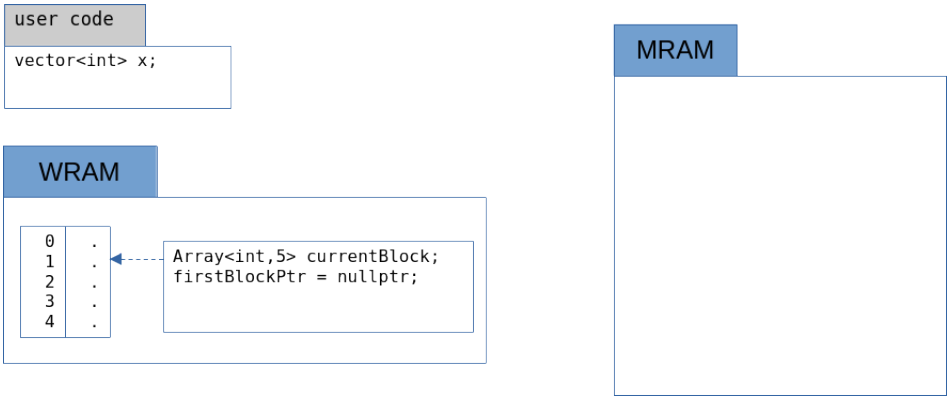
```
template<class T>
class vector {
public:
private:
    T cache[CACHE_NB_ITEMS];
};
```

Accessing an element at a given index is done through the operator[] operator:

```
template<class T>
class vector {
public:
    T operator[] (size_t idx)  {
        // if in cache, we can return the result.
        // otherwise, one must first request the MRAM
        // for filling the cache with the portion of the
        // vector holding the required index
        }
private:
    array<T,CACHE_NB_ITEMS> currentBlock;
};
```

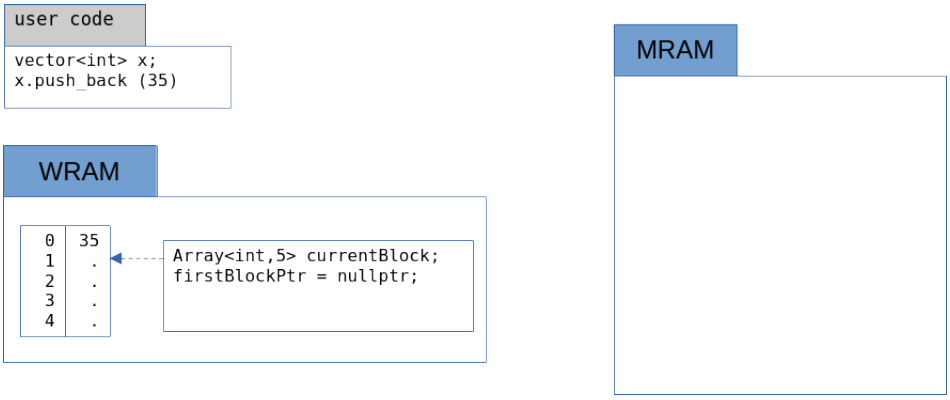
If the required index is not in the cache, we must reload the cache with the part of the vector containing the required index. This implies that the vector must have a structure that allows for this search.

In the earlier versions of the BPL, a natural structure was to manage a linked list of blocks of elements, each block being the same size as the cache, i.e., CACHE_NB_ITEMS elements. Let's take an example to clarify the concept:

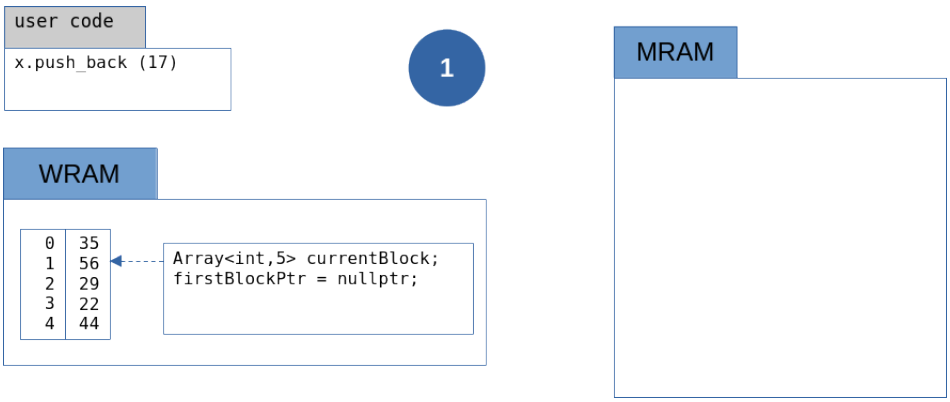


The vector x is created, with a cache size of 5 elements.

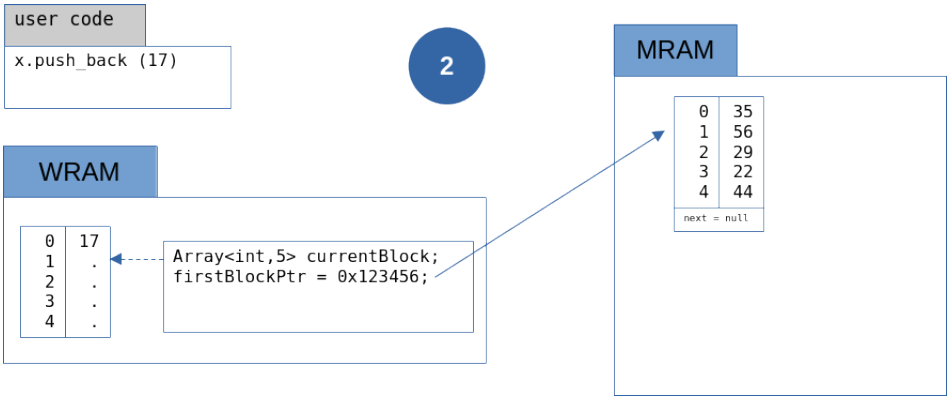
Next, an element is added using push_back; the cache in WRAM is updated to include this new element:



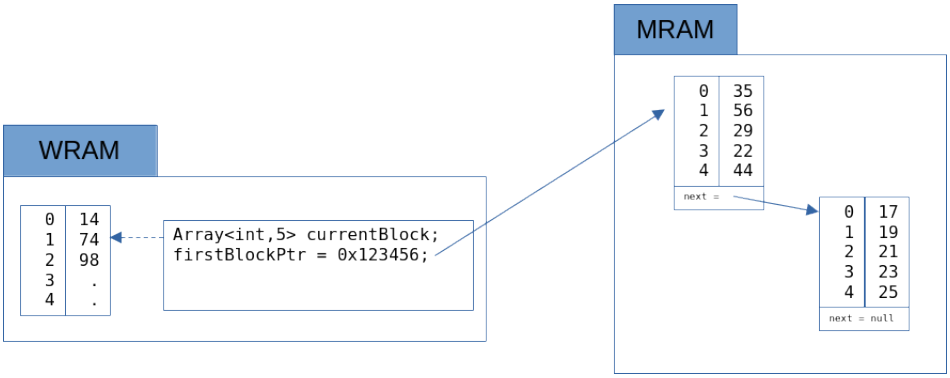
We continue by adding more elements until the cache is full:



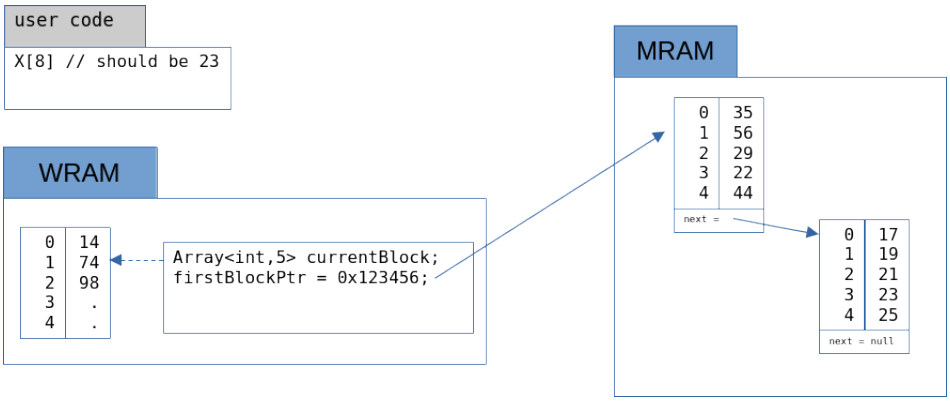
When we want to add one more element, we first need to dump the cache to MRAM:



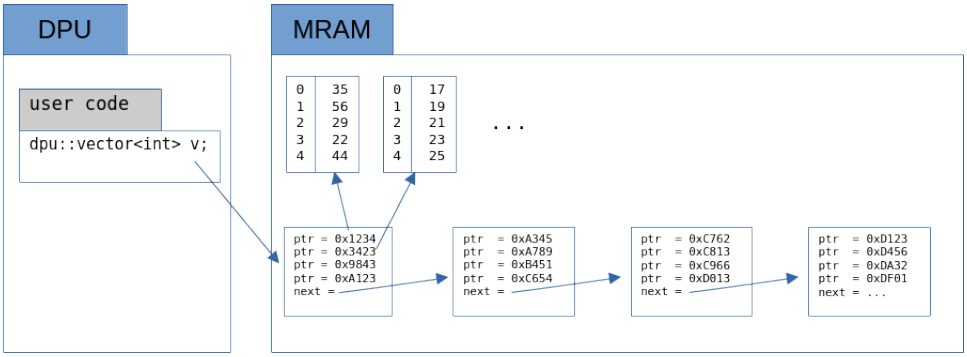
As we continue adding more elements, new blocks will be added to MRAM, maintaining a linked list structure between these different blocks:



Now, suppose we want to access an element of the vector at a given index using the operator[] function. We can thus use the linked list of blocks to retrieve the required block given its index:



However, this is not ideal for large vectors, as random access becomes increasingly costly as the lookup index grows.

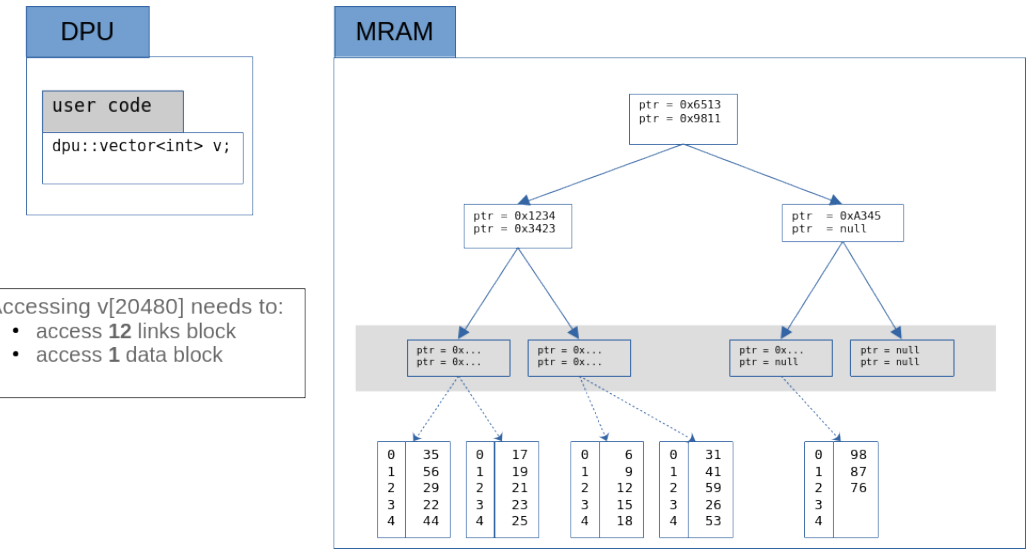


one data block → 5 items
one links block → 4 pointers

Accessing v[4] needs to:
- access **1** links block
- access **1** data block

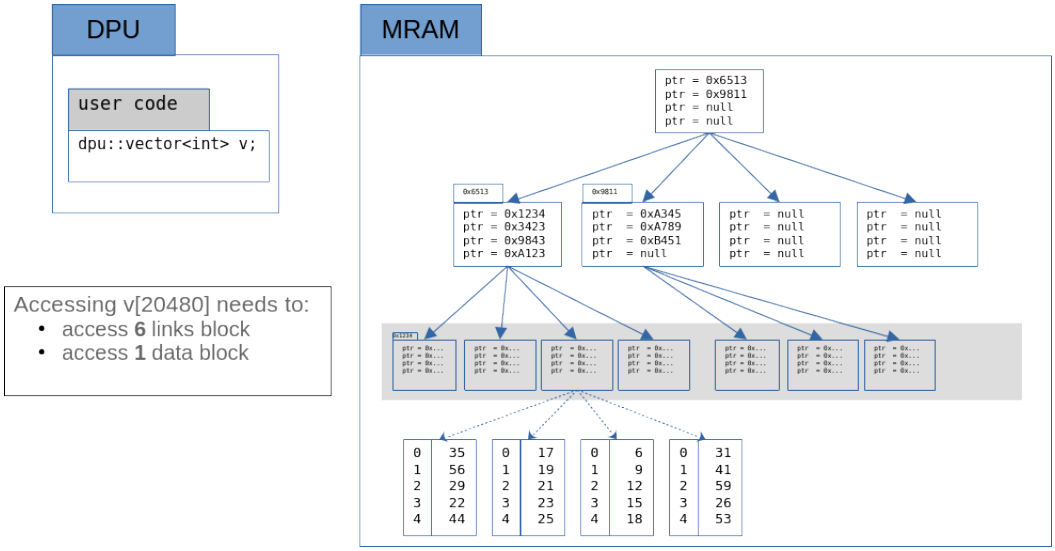
Accessing v[20480] needs to:
- access **1024** links block
- access **1** data block

We thus reach the limitation of a linked list, where searching for the desired block will take $O(N)$. A better alternative was to implement in the BPL a binary tree structure that allows for $O(\log(N))$ search.



Accessing v[20480] needs to:
- access **12** links block
- access **1** data block

The structure has even been improved to handle trees with more than two nodes per level. This structure is implemented by the MemoryTree class. The bpl::vector class therefore has an attribute of this class to manage such a tree.



Technically, the MemoryTree class is a template class:

```
template <
    typename ALLOCATOR,
    int NBITEMS_PER_BLOCK_LOG2,
    int MAX_MEMORY_LOG2
>
class MemoryTree;
```

whose parameters are:

- ALLOCATOR: a class that performs dynamic allocations in MRAM
- NBITEMS_PER_BLOCK_LOG2: number of nodes per level
- MAX_MEMORY_LOG2: memory size allocated to an instance of MemoryTree in WRAM. This indirectly implies a maximum number of nodes per instance of MemoryTree (and thus, indirectly, a maximum number of elements for bpl::vector).

Each leaf of this tree contains a list of addresses of blocks of elements from the vector. When iterating over the elements of a vector, it is the leaves of a MemoryTree object that contain the addresses of the blocks of elements to iterate over.

The implementation of the MemoryTree class is quite subtle, as it needs to be not only efficient but also use as little memory (WRAM) as possible. This class has a comprehensive suite of unit tests, independent of its use within the bpl::vector class.

2.8.3 The vector_view class

Sometimes, we need to provide a vector as an input parameter for the task, without the need to add new elements. In such cases, it is possible to not use a MemoryTree because the data of the vector broadcasted from the host to the DPU will be contiguous in MRAM. In this case, the vector only needs to know the MRAM address of the beginning of the data in order to either iterate over the elements or access a given index. This specific type

of vector, immutable and therefore not requiring a MemoryTree attribute, is implemented by the bpl::vector_view class.

In fact, bpl::vector_view and bpl::vector share a number of elements, such that in the end, it was possible to make bpl::vector inherit from bpl::vector_view. Ultimately, the specific features of bpl::vector_view result in its performance being better than that of bpl::vector.

2.8.4 Cache size tuning

One of the challenges of the UPMEM architecture is the limited capacity (64 KB) of the working memory, the WRAM. Moreover, this size is shared among the 16 tasklets, which ultimately results in an execution stack of 4 KB per tasklet.

The internal cache of an object of the vector class is allocated on the execution stack, so it is clear that the size of this cache must remain reasonable in order to leave space on the execution stack for other objects. Obviously, the problem becomes even more critical when multiple vectors are used. Cache sizing is therefore an important consideration.

Before going further, here are the different template parameters of bpl::vector:

```
template<
    typename T,
    typename ALLOCATOR,
    typename MUTEX,
    int MEMORY_SIZE_LOG2,
    int CACHE_NB_LOG2,
    bool SHARED_ITER_CACHE,
    int MEMTREE_NBITEMS_PER_BLOCK_LOG2,
    int MAX_MEMORY_LOG2
> vector {};
```

- T: the type of the elements in the vector
- ALLOCATOR: the type of the dynamic memory allocator (MRAM)
- MEMORY_SIZE_LOG2: the indicative size (log2) that all instances of bpl::vector should occupy in the execution stack
- CACHE_NB_LOG2: the number (log2) of internal caches; indeed, more than one cache may be needed (cf. the use of the swap function)
- SHARED_ITER_CACHE: a boolean indicating whether the internal cache can be used by an iterator of this vector

The last two parameters were defined earlier during the presentation of MemoryTree.

In order to simplify the use of the bpl::vector class, the *BPL* makes certain choices at compile time regarding the size of the internal cache. These choices are made based on the following criteria:

- A default value is defined for MEMORY_SIZE_LOG2 (typically 8), which means that the bpl::vector class should occupy 256 bytes in the execution stack of a tasklet for all instantiated objects.

- The size is adjusted based on the memory size of the type T; in other words, we calculate VECTOR_MEMORY_SIZE_LOG2 - Log2<sizeof(T)>::value.
- We also need to account for the number of vectors present in the prototype of the operator() of the task. Therefore, the value is divided by the number of vectors. Note: only the vectors present in the prototype can be detected using metaprogramming techniques; vectors declared inside the task body cannot be detected in this way.

These various calculations at compile time allow the developer to avoid doing them manually. While these calculations usually result in reasonable values, there may be cases where the developer might want to propose values they deem more suitable. We will soon see the mechanism that allows overriding these automatic calculations.

The use of the constants SHARED_ITER_CACHE and CACHE_NB_LOG2 will be described in the following sections.

2.8.5 Iterators

One of the main advantages of a proper vector class is the ability to easily iterate over its contained elements.

In the context of the UPMEM memory model, using an iterator presents a problem. Indeed, unlike a std::vector, an object of the bpl::vector class does not reside entirely in WRAM[4], and only the portion of the vector present in the internal cache is accessible at any given time.

Let's imagine that at a given time, the cache contains elements with indices 8 to 16 (exclusive). If we want to iterate over the vector from the beginning, we are left with two options:

1. We invalidate the current cache in order to fill it with the elements from indices 0 to 8 (exclusive). This allows iterating over the first 8 elements, and then a new cache will be loaded, until the iteration is completed. This means the iterator reuses the vector's internal cache. The disadvantage is that by the end of the iteration, the vector's internal cache is in a state that no longer matches its original state before the iteration started. The advantage is that it avoids needing a separate cache for the iterator, which results in a saving of WRAM, a scarce resource.
2. We can also ensure that the iterator object has its own cache, which prevents modifying the vector's internal cache, at the cost of using more WRAM.

The first solution is interesting to minimize WRAM usage, but the developer must be fully aware of its limitations. For example, if at least two iterators are needed on the same vector, the first solution is not feasible, and only the second one will yield correct results.

Currently, the first option is chosen by default by the *BPL* for memory usage reasons, but note that in some cases, the *BPL* might automatically switch to the second solution.

The configuration of this choice is made through the constant SHARED_ITER_CACHE, with 'true' meaning that an iterator will reuse the vector's internal cache, and 'false' meaning that a specific cache will be associated with the iterator. We will soon see how the developer can influence this constant.

[4]this alone shows that bpl::vector does not comply with the C++ standard, which requires a vector to be contiguous in memory

2.8.6 Swapping items

Some algorithms that use a vector sometimes need to swap two elements at indices i and k. A typical example is a sorting algorithm, e.g., selection sort.

```
1  template<class ARCH>
2  struct SortSelection {
3      decltype(auto) operator() (vector<uint32_t>& v) const {
4          for (size_t i=0; i<v.size(); i++)  {
5              size_t k=i;
6              value_type x = v[i];
7              size_t j=i+1;
8              for (auto it2=v.begin()+j; it2!=v.end(); ++it2, ++j)  {
9                  if (*it2<x) {  k=j;  x=*it2;  }
10             }
11             swap (v[i], v[k]);
12         }
13         return v;
14     }
15 };
```

We can see on line 11 the call to the swap function to exchange the elements of vector v at indices i and k.

However, we understand that this will cause issues with the design of bpl::vector as we've presented it so far. Indeed, in general, there is no guarantee that both indices i and k will be present in the vector's internal cache. For example, let's take i = 5 and k = 21. If the cache contains elements with indices in the range $[0 : 8[$, the element $v[i]$ will be in the cache, but not the element $v[k]$. We would therefore need to load the elements with indices in the range $[16 : 24[$ into the cache, but this would cause the element at index 8 to no longer be available in the cache. The conclusion is simple: the ability to swap two elements requires the vector to have two internal caches in the case where each element to be swapped is in a different cache.

Still for optimization reasons, the bpl::vector class is configured by default with a single cache. However, if an attempt is made to use the swap function, a compilation error will be generated with an error message indicating that only one cache is available.

To configure the *BPL* so that bpl::vector can use swap (with two internal caches), you should write the following:

```
1  template<class ARCH>
2  struct SortSelection : bpl::Task<ARCH>  {
3
4      struct Config  { static constexpr bool SWAP_USED = true; };
5      USING(ARCH,Config);
6
7      decltype(auto) operator() (vector<uint32_t>& v) const {
8          /* same as before */
9      }
10 };
```

The difference occurs at lines 4 and 5: we define a Config structure that sets the constant SWAP_USED to 'true' in order to force the use of a second internal cache, then we associate this configuration with the architecture via USING(ARCH,Config). This forms the configuration mechanism of the *BPL* , which we will revisit later.

2.8.7 Performance

In fact, the UPMEM SDK allows accessing an array element located in MRAM as if it were in WRAM. In a way, a cache similar to what we saw for bpl::vector and bpl::vector_view is used. However, this is a cache for a single element, so while it makes code writing easier, it is not optimal in terms of performance[5].

We propose to compare 3 cases in a scenario of writing elements to an array:

- handcrafted_write: a handwritten version that minimizes WRAM/MRAM exchanges
- umpem_write: a version that leverages the UPMEM cache
- vector_write: a version that uses bpl::vector

Here are the three associated implementations:

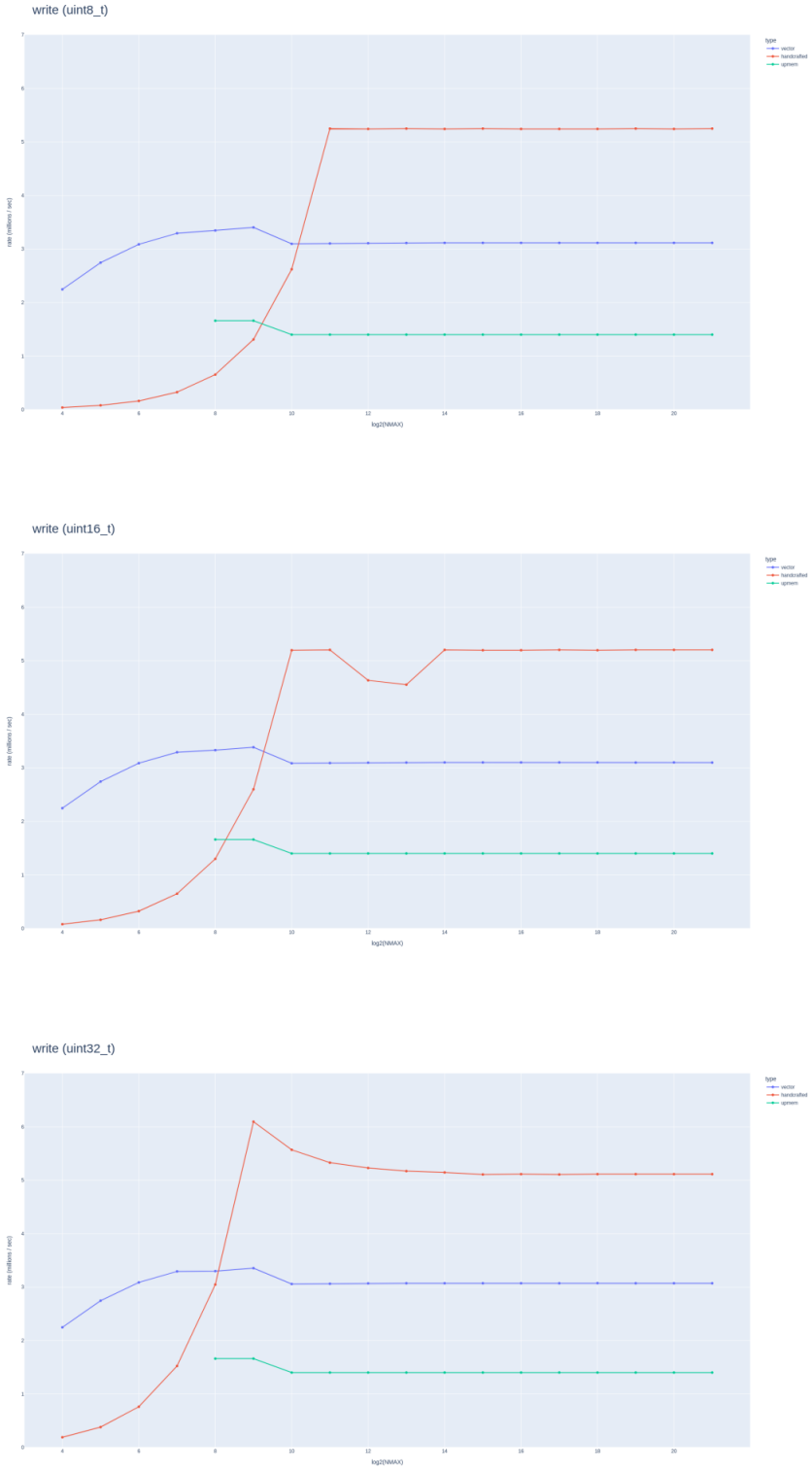
```
void handcrafted_write () {
    __dma_aligned type input[BUFFER];
    for (uint32_t n=0; n<NMAX; n+=BUFFER) {
        for (uint32_t i=0; i<BUFFER; i++)  {  input[i]  = type(n+i); }
        mram_write (
           input,
           (__mram_ptr void *) ( (type*)big + n), sizeof(input));
    }
}
```

```
type __mram_noinit  big [NMAX];
void upmem_write ()  {
    for (uint32_t n=0; n<NMAX; n++)  {  big[n] = type(n);  }
}
```

```
void vector_write ()  {
    bpl::core::vector<type> vec;
    for (uint32_t n=0; n<NMAX; n++)  {  vec.push_back (type(n));  }
```

Here are the results for different types of elements:

[5] which is also clearly stated in the UPMEM documentation



When the number of items grows, we can see that bpl::vector is positioned between the optimal version (handcrafted_write) and the version with the UPMEM cache. This represents a rather positive result for bpl::vector because it also has to account for dynamic allocation; in the other two cases, the number of elements is fixed, while in the bpl::vector case, the number of elements is not initially known, i.e., elements are added one by one via push_back.

2.9 Customization of the BPL

The *BPL* aims to make the developer's life easier by making decisions at compile time. However, it should be kept in mind that the *BPL* is designed to be a generic library applicable to a wide range of use cases. As a result, the choices made may not be optimal in certain scenarios. Therefore, it is important for the *BPL* to offer a configuration mechanism so that the developer can adjust, if needed, certain decisions automatically made by the library.

A number of constants, which are normally calculated at compile time, can be defined by the developer. It should be noted that these constants are applicable only when the target architecture is UPMEM.

The mechanism for configuring the *BPL* has already been discussed in a previous section: we define a structure containing certain constants, then associate this structure with the architecture at the task level. This specific configuration is thus tied to the task and not to the launcher. Let's revisit the previous example, with additional configuration constant definitions:

```
template<class ARCH>
struct SortSelection : bpl::Task<ARCH>  {

    struct Config  {
        static constexpr bool SWAP_USED              = true;
        static constexpr bool SHARED_ITER_CACHE      = false;
        static constexpr bool VECTOR_SERIALIZE_OPTIM = true;
    };

    USING(ARCH,Config);

    decltype(auto) operator() (vector<uint32_t>& v) const {
        /* same as before */
    }
};
```

It should be noted that the Config structure could be defined outside the structure defining the task (SortSelection in the example). However, since the configuration is task-specific, it makes more sense to define it within the task definition. One could even imagine a configuration where a constant depends on the ARCH parameter.

The list of constants that the developer can use is described in the Application Programming Interface section of this document.

2.10 The CMake toolchain

Any library comes with a way to use it, not only from the perspective of its API but also regarding the toolchain that supports it, in order to generate resources (often binaries) based on the library. Moreover, a library should be accompanied by a suite of tests (unit tests, benchmarks, ...) to verify its quality; once again, the chosen toolchain must be able to easily integrate the generation of such tests.

The toolchain used by the *BPL* is CMake mainly because it is a widely adopted and well-maintained tool.

More specifically, one of the challenges with the UPMEM architecture is having to generate two binaries, one for the host side and the other for the DPU side, a task that CMake can easily handle. It is also thanks to CMake and its configure_file command that it was possible to define a generic DPU source code, one of the cornerstones of the *BPL* .

Another aspect of the *BPL* , seemingly trivial at first glance, required some work at the CMake level; it concerns the management of the global tag, which results in the need to perform an initial phase of DPU binary compilation in order to benchmark the execution stack size of the tasklets.

Finally, CMake enabled the creation from scratch of minimalist *BPL* projects via the NEWPROJECT_NAME variable.

2.11 Metaprogramming in the BPL

As we have seen so far, the *BPL* performs numerous actions at compile time, allowing it to, among other things, handle data serialization, calculate certain constants to optimize WRAM usage, manage tags to add semantics when needed, etc...

All these actions require the application of metaprogramming techniques in C++. The design history of the *BPL* shows certain evolutions in these techniques; for example, the shift to using the C++20 standard took place midway through the project, with some simplifications, such as the use of C++ concepts.

A set of type traits has also been implemented for the needs of the *BPL* (see the metaprog.hpp file), but it is entirely possible that some of the implementations were already present in the C++ standard library. It should also be noted that at one point, the hana metaprogramming library was considered, but this option was ultimately not pursued.

Some general remarks:

- The serialization part of the *BPL* requires analyzing the prototype of the task to be executed, so there is a whole set of type traits enabling this analysis.
- Another point regarding serialization: since the developer can create their own POD types, it is important to be able to iterate over the different attributes of a structure. However, this remains difficult in C++ because there is no built-in introspection support (at least until C++26). Some techniques to mitigate this limitation have been used in the *BPL* (e.g., class_fields_call), but some limitations may still persist.
- The use of std::tuple is ubiquitous, and many type traits have been added to simplify the development of the *BPL* . As an example, handling the global tag requires certain contortions based on std::tuple.
- The management of constants used to configure the *BPL* also relies on certain type traits. This is even a case where using the macro processor proved to be essential (see DEFINE_GETTER).
- It is worth noting that the most commonly used C++ metaprogramming technique in the *BPL* is certainly class template specialization. Such a technique also opens the door to future improvements through the introduction of new specializations.

Chapter 3

Testing the BPL

3.1 Unit tests

3.1.1 Introduction

Unit tests represent an important part of a library: they make it possible to (a) verify that the available features produce the expected results, and (b) detect potential regressions in a new version of the library. The second point is particularly important when significant changes have been made to the source code in a new version.

Consequently, the number of unit tests may increase from one version to the next to cover as many aspects as possible. In other words, the number of unit tests should never decrease over time.

Even more importantly, existing tests should not be modified over time, except in very specific cases, such as an API change, and only when absolutely necessary.

3.1.2 Tests categories

For the *BPL* , the unit tests are generally categorized as follows:

- TestLauncher: tests for the bpl::Launcher class
- TestMemoryTree: tests for the MemoryTree class ("directly" tested, ie. not in the context of its regular usage inside a bpl::vector)
- TestMetaprog: tests for some C++ metaprogramming tools
- TestMisc: miscellanous tests like global and once tags
- TestSerialize: tests about serialization of objects.
- TestSketch: tests that compare sketches
- TestSort: tests for sorting vectors
- TestSplit: tests for the split BPL concept, ie. something that split input data as a parallelization scheme
- TestSynchro: tests for synchronizing data access (e.g. mutex)

- TestVector: tests for using bpl::vector

Some tests are run using a launcher configured for either the *UPMEM* architecture or a multicore architecture, while others do not require a launcher (see, for example, the TestMetaprog).

Note that we will not detail all the unit tests here. Readers can directly browse the source code to get an overview of the content of each test.

3.1.3 Unit test framework

The *BPL* uses the Catch2 unit testing framework because it is popular and actively maintained on GitHub.

The *BPL* uses Catch2 in a straightforward way, so there is nothing specific to add here. Readers can consult `https://github.com/catchorg/Catch2` for further details if needed.

Here is a minimal test file using Catch2 in the context of the *BPL* :

```
#include <common.hpp>
#include <tasks/VectorCreation.hpp>

using namespace bpl;

TEST_CASE ("VectorCreationSimple", "[Vector]" )
{
    size_t nbErrors1 = 0;
    size_t nbErrors2 = 0;
    size_t nbitems = 1<<15;
    size_t n=0;

    Launcher<ArchUpmem> launcher {1_dpu};

    for (auto const& res : launcher.run<VectorCreation>(nbitems))  {
        nbErrors1 += res.size() == nbitems ? 0 : 1;
        size_t k=0;
        for (auto x : res) { nbErrors2 += x==(k+n) ? 0 : 1;   k++; }
        n++;
    }
    REQUIRE (nbErrors1 == 0);
    REQUIRE (nbErrors2 == 0);
}
```

A few remarks:

- Include the common.hpp file, which provides a minimal set of includes for using the *BPL* .
- Include the header file of the task to be tested; in this example, VectorCreation.
- Define a VectorCreationSimple test case where:
  - Declare a bpl::Launcher.

- Run the VectorCreation task through the launcher.
- Iterate over each result generated by the call to run and check the content against the expected result.
- Use the REQUIRE command to verify that everything is correct.

Almost every unit test follows this pattern.

Sometimes, a test is run with different inputs. This allows a deeper verification of the task, but only in the context of unit tests. Here, we are concerned only with whether the test passes or fails relative to the expected result; performance is not considered at this stage and will be addressed separately in the benchmark test suite.

3.1.4 Source code hierarchy

As mentioned earlier, the *BPL* relies on CMake as its build system. For the unit tests, the file structure is as follows:

- **host**: directory containing tests written using the Catch2 framework.
- **tasks**: directory containing the tasks (one file per task).

To test a new task, MyTask, follow these steps:

- Implement the task in a MyTask.hpp file and place it in the tasks directory.
- In tasks/CMakeLists.txt, add the name of the new task to the TASKS_LIST variable.
- Add a TEST_CASE in a file within the host directory to implement the unit test. The task header (e.g., tasks/MyTask.hpp) must be included at the beginning of the file.

3.1.5 Building and running the unit tests

The unit tests are built using the bpl-unittests.host target in the Makefile generated by CMake. From the build directory, you can generate the binary with:

```
make bpl-unittests.host
```

You can then run the unit test suite with:

```
./test/unit/host/bpl-unittests.host -d y
```

The -d y option is optional; it displays each test name along with its execution time.

3.2 Benchmark tests

3.2.1 Introduction

The main idea of the BIOPIM project is to implement algorithms on PIM architectures (e.g., *UPMEM*) and evaluate the potential speedup. It is therefore important to be

able to compare execution times against a more standard approach, for example using multithreading.

A key feature of the *BPL* is the ability to write a single source code that can run on different architectures, such as multicore and *UPMEM* . This allows us to directly compare the execution times of the exact same algorithm implementation across different architectures. For example, we could write the following:

```
{
  auto t0 = timestamp()
  Launcher<Multicore>{}.run<SomeAlgorithm> (data);
  auto t1 = timestamp()
  Launcher<Upmem>{}.run<SomeAlgorithm> (data);
  auto t2 = timestamp()
}
```

Here, $t1 - t0$ represents the execution time of the algorithm on a multicore architecture, while $t2 - t1$ represents the execution time on a *UPMEM* architecture. This makes it possible to generate benchmark reports showing both the scalability of a given architecture and the speedup of one architecture (e.g., *UPMEM*) compared to another (e.g., multicore).

In practice, using the *UPMEM* part of the *BPL* (and the UPMEM SDK under the hood) introduces some overhead, such as broadcasting data between the host and the DPUs—similar to sending data over a network. Benchmarking the *BPL* is therefore important to quantify the cost of these overheads. The *BPL* can generate statistics about these overheads, allowing experts to assess whether a particular overhead is significant.

The tests used for benchmarking the *BPL* are a subset of the unit tests, but with different input data. This approach has the advantage that no additional tests need to be written specifically for benchmarking, since the unit tests already cover the functionality. Keep in mind, however, that during benchmarking, we do not verify the correctness of the results—we are only interested in execution times. Correctness is ensured during the unit testing stage.

Benchmark reporting requires much more information than unit testing, where we only need to know whether a test passes or fails. During benchmarking, we may need to generate various types of visualizations to assist the expert. The *BPL* provides a Jupyter notebook that takes as input the benchmark tests output and produces a PDF report containing multiple graphics, which we will now describe.

3.2.2 The benchmark report

The Jupyter notebook to be launched is located in test/benchmark/notebooks and is named 1.Benchmark.ipynb. It takes as input the traces generated during the execution of the benchmark.

After some introductory text, a section named after each test is generated with the following sub-sections:

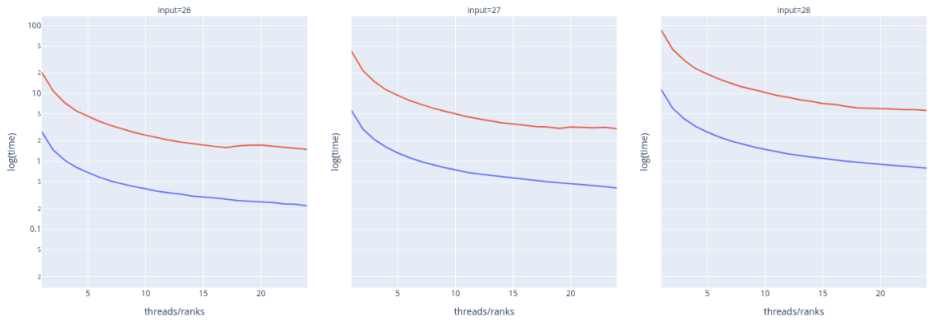
- **Description**: a short description of the test. This description should be written as a comment in the test source code following specific metadata rules.
- **Remarks**: additional comments or notes, also included as comments in the source code.

- **Benchmark-input**: defines the input range, i.e., the values for which the test will be executed.

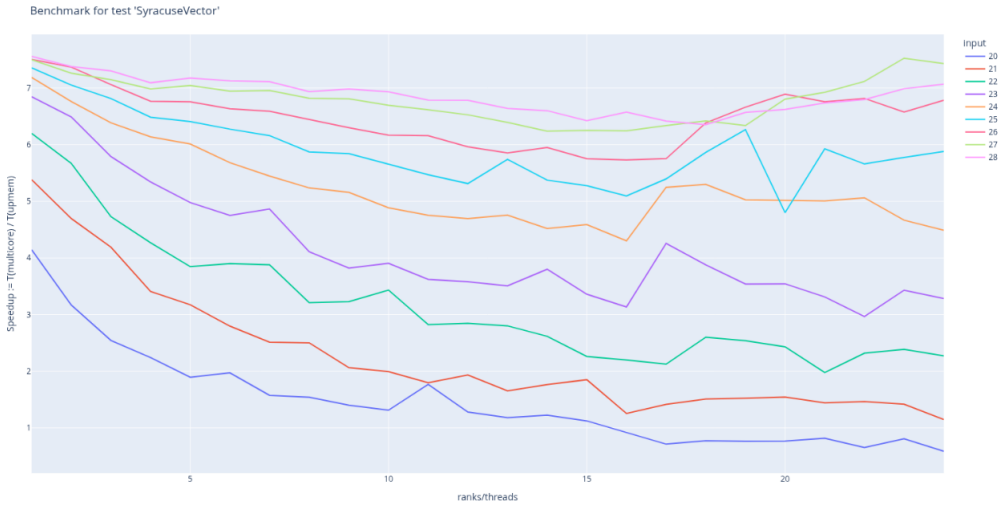
- **Benchmark-split**: indicates whether an input argument has been "split" among process units.

- **Source code**: the C++ source code of the test. Including the code in the report is useful, but it is recommended to avoid excessively long snippets.

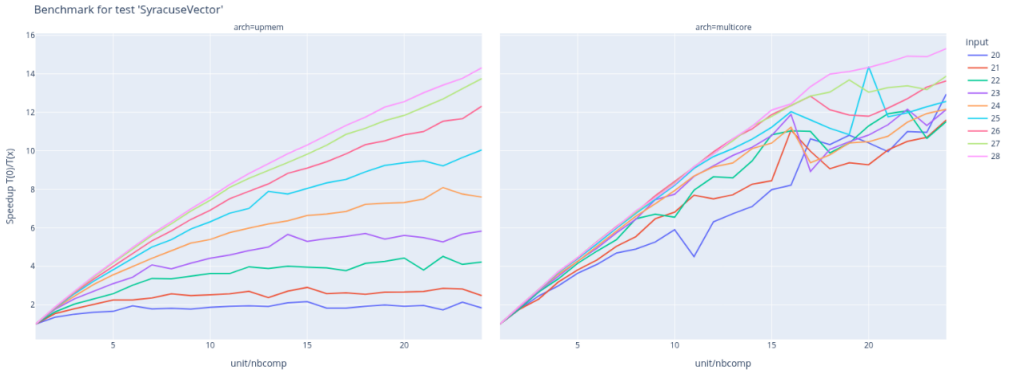
- **Multicore vs *UPMEM*** : compares execution times on a logarithmic scale. One graph is generated per input value. The x-axis represents threads in the case of multicore architectures and ranks in the case of UPMEM.



- **Speedup *UPMEM* vs Multicore**: the ratio of execution time for 1 thread on multicore divided by the execution time for 1 rank on *UPMEM* .

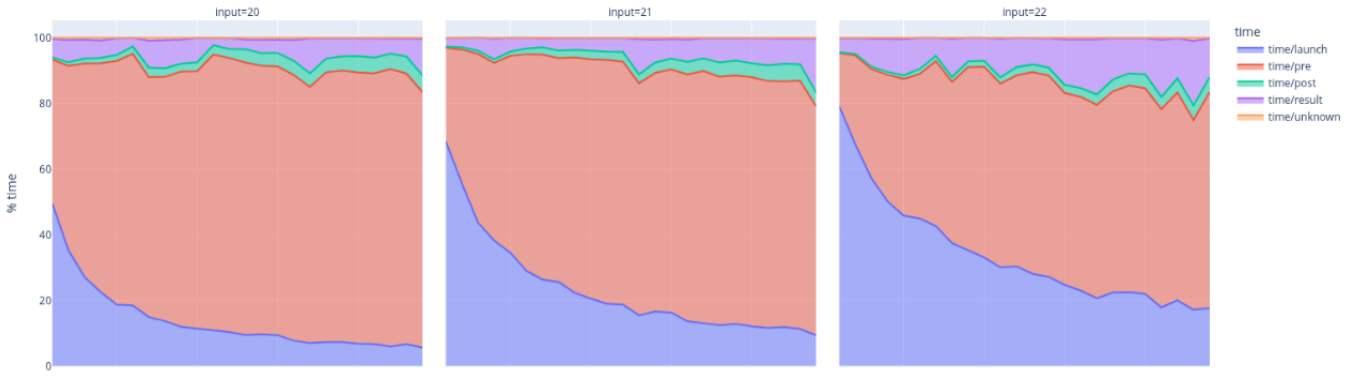


- **Scalability per architecture**: for both multicore and *UPMEM* , this section displays scalability per process unit. For a given number N of process units (threads or ranks, ie. 16*64*tasklets per rank), the ratio of the execution time for 1 process unit to that for N process units is shown.



- **Overheads for the *UPMEM* architecture**: the *BPL* provides statistics for overheads specific to *UPMEM* . These timings are measured from the host's perspective, not the DPU's.
  - **launch**: execution time of the task, i.e., the execution of the dpu_launch function (with DPU_SYNCHRONOUS argument).
  - **pre**: preparation and broadcasting of the input data from the host to the DPU (including call to scatter/gather function dpu_push_sg_xfer)
  - **post**: retrieval of output metadata from the DPU to the host (call to dpu_push_xfer function). This metadata is used to reconstruct the task result.
  - **result**: preparation and transmission of the task's output data back to the host (calls to dpu_copy_from_mrams for instance)
  - **unknown**: parts that cannot be measured in the previous categories (e.g., destructors) are included here.

  In the following example, preparing and sending the input data to the DPU takes a significant amount of time compared to the actual processing.



The execution times recorded during benchmarking are summarized in tables at the end of the report. While this may appear verbose, it can be very useful for referencing specific values from previous benchmark runs.

3.2.3 Annotation of a test

As mentioned earlier, the source code of a benchmark test must include a few annotations that are used to generate the benchmark report. Each annotation should be written inside

a C++ comment. The annotation label begins with the @ character (e.g., @description), followed by a : and a free text describing the annotation. Here is a complete example of a test source file with benchmark annotations:

```
////////////////////////////////////////////////////////////////////////
// @description: Takes a vector as input, iterates its content and
// compute the checksum.
// @benchmark-input: 2^n for n in 20,22,24,26,28
// @benchmark-split: yes
////////////////////////////////////////////////////////////////////////
template<class ARCH>
struct VectorChecksum : bpl::Task<ARCH>
{
    USING(ARCH);

    auto operator() (vector<uint32_t> const& v)
    {
        uint64_t checksum = 0;
        for (auto x : v)  {  checksum += x;  }
        return checksum;
    }
};
```

3.2.4 Building and running the benchmark tests

The benchmark tests are built using the bpl-benchmark.host target in the Makefile generated by CMake. From the build directory, you can generate the binary with:

```
make bpl-benchmark.host
```

You can then run the benchmark test suite with:

```
./test/benchmark/host/bpl-benchmark.host
```

During the tests, traces are generated. These traces serve as input to the Jupyter notebook, so they need to be redirected to a file. It is possible to use the nohup command to run the tests and redirect the traces to a file whose name includes the current Git commit hash:

```
nohup ./test/benchmark/host/bpl-benchmark.host \
 > ../test/benchmark/notebooks/traces/traces_`git rev-parse  HEAD`.txt &
```

By following this recommendation, it is possible to preserve benchmark traces for a given Git version, allowing a benchmark report to be regenerated later for an older version of the library.

3.2.5 Other remarks

Creating a benchmark test suite is not straightforward, as there is no clear rule for deciding when to add a test. A good rule of thumb is to be fair with respect to the library's features:

include tests where the library performs well on the *UPMEM* architecture compared to multicore (e.g., small data / heavy computation), but also include cases where *UPMEM* overheads are significant (e.g., large data / light computation).

For example, SketchJaccardDistance compares all possible pairs of sketches, resulting in a triple loop scanning the data. This is a scenario that favors *UPMEM* because the broadcast cost is small compared to the total computation time. On the other hand, sorting a vector with an $O(n \log n)$ algorithm is likely faster on multicore, because on *UPMEM* the broadcasts (host $\rightarrow$ DPU for input and DPU $\rightarrow$ host for output) can take a significant amount of time relative to the sorting itself. Therefore, it is important to select tests for benchmarking based on the computational complexity relative to the data size to be broadcast.

Another consideration is the total execution time of the benchmark suite. Since the same test is run with multiple input parameters, execution times can grow quickly. It is therefore important not to include too many tests, or the benchmark could take an excessively long time to complete.

3.2.6 Future improvements

Each new version of the BPL should commit the raw data generated during benchmark execution. This makes it possible to compare two versions and identify any improvements in execution time. A dedicated Jupyter notebook can be created to visualize the differences between the two versions using graphics.

3.3 The LOC metric

So far, we have tested the BPL through unit tests and benchmarks. There is an additional way to evaluate the BPL, from the developer's perspective, in cases where one wants to use the UPMEM architecture: does the BPL allow for faster development compared to using the UPMEM SDK?

To ensure a fair comparison, we will assume here that the developer has equivalent knowledge of (1) the UPMEM SDK and (2) the BPL API; in other words, that they possess sufficient understanding of both APIs to write code directly using either approach. [1]

Therefore, the metric that comes to mind is the number of lines of code required to write a program, either using the UPMEM API or the BPL API. From now on, this metric will be referred to as LOC, for "Lines Of Code."

Of course, it is not practical to write every program using both approaches in order to compare the LOC, since the goal is primarily to develop using the BPL API. However, it is still interesting to perform this exercise on a few examples.

3.3.1 Example

We will start with the task of calculating the checksum of a buffer, as this is an example provided in the UPMEM documentation. We can therefore simply write the BPL version and compare the LOC in both cases.

[1] In practice, the learning curve for the UPMEM SDK remains more complex than that of the BPL.

Here is the UPMEM SDK version. First the DPU part (with all comments removed):

```
#include <mram.h>
#include <stdbool.h>
#include <stdint.h>
#define CACHE_SIZE 256
#define BUFFER_SIZE (1 << 16)
__mram_noinit uint8_t buffer[BUFFER_SIZE];
__host uint32_t checksum;
int main() {
  __dma_aligned uint8_t local_cache[CACHE_SIZE];
  checksum = 0;
  for (unsigned int bytes_read = 0; bytes_read < BUFFER_SIZE;) {
    mram_read(&buffer[bytes_read], local_cache, CACHE_SIZE);
    for (unsigned int byte_index = 0; (byte_index < CACHE_SIZE)
         && (bytes_read < BUFFER_SIZE); byte_index++, bytes_read++) {
      checksum += (uint32_t)local_cache[byte_index];
    }
  }
  return checksum;
}
```

and the host part

```
#include <dpu.h>
#include <assert.h>
#include <stdint.h>
#include <stdio.h>
#ifndef DPU_BINARY
#define DPU_BINARY "trivial_checksum_example"
#endif
#define BUFFER_SIZE (1 << 16)
void populate_mram(struct dpu_set_t set, uint32_t nr_dpus) {
  struct dpu_set_t dpu;
  uint32_t each_dpu;
  uint8_t *buffer = malloc(BUFFER_SIZE * nr_dpus);
  DPU_FOREACH(set, dpu, each_dpu) {
    for (int byte_index = 0; byte_index < BUFFER_SIZE; byte_index++) {
      buffer[each_dpu * BUFFER_SIZE + byte_index] = (uint8_t)byte_index;
    }
    buffer[each_dpu * BUFFER_SIZE] += each_dpu;
    DPU_ASSERT(dpu_prepare_xfer(dpu, &buffer[each_dpu * BUFFER_SIZE]));
  }
  DPU_ASSERT(dpu_push_xfer(set, DPU_XFER_TO_DPU, "buffer", 0, BUFFER_SIZE,
          DPU_XFER_DEFAULT));
  free(buffer);
}
void print_checksums(struct dpu_set_t set, uint32_t nr_dpus) {
  struct dpu_set_t dpu;
```

```
  uint32_t each_dpu;
  uint32_t checksums[nr_dpus];
  DPU_FOREACH(set, dpu, each_dpu) {
    DPU_ASSERT(dpu_prepare_xfer(dpu, &checksums[each_dpu]));
  }
  DPU_ASSERT(dpu_push_xfer(set, DPU_XFER_FROM_DPU, "checksum", 0,
          sizeof(uint32_t), DPU_XFER_DEFAULT));
  DPU_FOREACH(set, dpu, each_dpu) {
    printf("[%u] checksum = 0x%08x\n", each_dpu, checksums[each_dpu]);
  }
}
int main() {
  struct dpu_set_t set, dpu;
  DPU_ASSERT(dpu_alloc(DPU_ALLOCATE_ALL, NULL, &set));
  DPU_ASSERT(dpu_load(set, DPU_BINARY, NULL));
  DPU_ASSERT(dpu_get_nr_dpus(set, &nr_dpus));
  populate_mram(set, nr_dpus);
  DPU_ASSERT(dpu_launch(set, DPU_SYNCHRONOUS));
  print_checksums(set, nr_dpus);
  DPU_ASSERT(dpu_free(set));
  return 0;
}
```

which totals $19 + 47$ lines, or 66 in all.

Now, using the BPL API, we have:

```
#include <bpl/core/Task.hpp>
template<class ARCH>
struct VectorChecksum : bpl::Task<ARCH>  {
    USING(ARCH);
    auto operator() (vector<uint32_t> const& v)  {
        return accumulate(v.begin(), v.end(), uint64_t{0});
    }
    static uint64_t reduce (uint64_t a, uint64_t b)  { return a+b; }
};
```

and

```
#include <vector>
#include <cstdio>
#include <bpl/bpl.hpp>
int main() {
    std::vector<uint32_t> v;
    for (size_t i=1; i<=1<<16; i++)  {  v.push_back(i);  }
    Launcher<ArchUpmem> launcher {1_dpu};
    printf ("checksum: %ld\n", launcher.run<VectorChecksum>(split(v)));
}
```

which totals $9 + 9$ lines, or 18 in all.

Thus, we have 66 LOC for the UPMEM SDK and 18 LOC for the BPL, a ratio of 3.66. Note that the ratio would be 3.88 if we count characters instead of lines.

In addition to the significant reduction in code size with the BPL, code written using the BPL is also much more readable than with the UPMEM SDK. Indeed, reading the BPL code makes it immediately clear that the task is to compute the checksum of a vector, which is far less obvious in the UPMEM SDK version, where all the low-level SDK calls reduce overall readability.

Of course, it would be worthwhile to extend this LOC comparison to other examples, but it seems fairly clear that the advantage would still favor the BPL. The remaining point to determine is the overhead of the BPL compared to writing directly using the UPMEM SDK. If a BPL version were consistently 50% slower than a SDK version, its usefulness could be limited. In this regard, benchmark tests can provide a typology of algorithms that are likely to benefit from execution via the BPL on a UPMEM architecture, and conversely, those that might be penalized.

Related to readability, it is also conceivable that a developer could implement a simple algorithm using the BPL to run on a UPMEM architecture without needing to know the UPMEM SDK, which would represent a significant benefit of the BPL.